

Министерство науки и высшего образования
Российской Федерации

КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Ю.М. ВИШНЯКОВ, Р.Ю. ВИШНЯКОВ

МЕТОДЫ РАСПОЗНАВАНИЯ СИМВОЛЬНЫХ
ПОСЛЕДОВАТЕЛЬНОСТЕЙ

Учебное пособие

Краснодар
2019

УДК 004.4'2(4)

ББК 32.81 я 73

В 58

Рецензенты:

Доктор технических наук, профессор

И.Ф. Астахова

Доктор технических наук, профессор

Я.М. Ерусалимский

Вишняков, Ю.М., Вишняков, Р.Ю.

В558 Методы символьных последовательностей: учебное пособие/ Ю.М. Вишняков, Р.Ю. Вишняков. – Краснодар: Кубанский гос. ун-т, 2019. – 107 с. 500 – экз.

ISBN

Изложены методы распознавания символьных последовательностей. Рассмотрены такие модели распознавателей, как детерминированная, недетерминированная, система переходов, а также их эквивалентные преобразования. Показана тесная взаимосвязь формальных грамматик и распознавателей, рассмотрены процедуры их взаимного преобразования. Обсуждены автоматы с магазинной памятью и неавтоматные методы распознавания и идентификации.

Адресуется студентам, изучающим дисциплину «Методы разработки трансляторов» бакалавриата направления подготовки 02.03.02 «Фундаментальная информатика и информационные технологии», спецсеминар магистратуры – направления подготовки 02.04.02 «Фундаментальная информатика и информационные технологии», а также дисциплину «Формальные грамматики, языки и методы компиляции» бакалавриата направления 02.03.01 «Математика и компьютерные науки» и дисциплины «Методы и средства автоматической обработки информации» и «Теория распознающих автоматов», «Теоретические основы и технологии информационного поиска» магистратуры направления 02.04.01 Математика и компьютерные науки»

УДК 004.4'2(4)

ББК 32.81 я 73

ISBN

© Кубанский государственный

© университет, 2019 Вишняков Ю.М., Вишняков

Р.Ю.

ВВЕДЕНИЕ

Символьная обработка данных сегодня одна из актуальных и востребованных областей информатики. Особую остроту придают ей нерешенные проблемы в области автоматизации обработки естественных языков, поскольку даже при достаточности вычислительных ресурсов трудности обуславливаются отсутствием универсальных моделей и инструментов описания и моделирования естественных языков. Объяснение данного факта кроется в сложности и неоднозначности естественных языков, порожаемыми отношениями синтаксиса (формы) и семантики (содержания).

По отдельным направлениям, в частности, компиляции языков программирования, которое выделилось в отдельное предметное направление, имеются достаточно неплохие результаты. В компиляции основную теоретическую базу составляют формальные грамматики и языки, методы синтаксического анализа и перевода, для которых отработаны основные инструменты символьной обработки, базирующиеся в основном на понятии автомата.

Сам автомат, зародившись в недрах цифровой и вычислительной техники, перерос ее рамки и превратился в одно из фундаментальных понятий информатики. В компиляции автоматы распознают цепочки символов на принадлежность некоторому множеству (языку) и на их основе строятся сканеры и текстовые процессоры. Они широко используются в лексическом, синтаксическом анализе и переводе благодаря следующим свойствам:

- возможность формализации процедуры разбора цепочек символов;
- простота эффективной программной реализации.

Процесс проектирования автомата получил названия синтеза и восходит к цифровой технике. Синтез подразделяется на два уровня – структурный и абстрактный. В цифровой технике практическое применение нашел структурный синтез, в то время как абстрактный синтез органически вошел составной частью в программную инженерию. Здесь наиболее распространена модель конечного автомата, называемая распознавателем языков, в ней реализуется двоичная функция выходов со значениями true и false.

Как правило, представляется такой автомат моделью автомата Мура с одним выходом.

Следует заметить, что распознавание в качестве частного случая символьной обработки, может реализовываться автоматами более высокого порядка, каковыми являются автоматы с магазинной памятью, а также оно может реализовываться не автоматными методами.

Цель, которую преследовали авторы настоящего учебного пособия, состоит в освещении фундаментальных аспектов распознавания символьных последовательностей (цепочек символов), связи распознавания с теорией автоматов и формальными грамматиками и языками.

Опираясь на накопленный многолетний опыт преподавания дисциплин, ориентированных на область программного обеспечения вычислительной техники, авторы попытались выделить символьную обработку отдельно вне конкретных учебных дисциплин, хотя ко многим из них она имеет самое прямое и непосредственное отношение. Также в большей степени авторы ориентировались на студенческую аудиторию и поэтому стремились выстроить материалы логически от простого к сложному, большее внимание уделив раскрытию физического существа понятий и положений в их взаимной связи, стремились избежать излишней формализации. Поэтому в пособии приводится много примеров, на которых строятся объяснения и рассуждения.

Само учебное пособие существенно улучшает и дополняет работу [5] и направлено на освещение теоретической базы проектирования трансляторов, хотя не охватывает ее в целом. Однако затрагиваемые в пособии вопросы могут составить предмет отдельного изучения и использоваться в качестве инструментов исследования и проектирования в тех областях программной инженерии, где используется символьная обработка.

Авторы будут признательны читателям за пожелания и замечания к работе.

1. ОСНОВНЫЕ ПОЛОЖЕНИЯ ТЕОРИИ ФОРМАЛЬНЫХ ГРАММАТИК

Рассмотрим некоторые базовые положения и понятия теории формальных грамматик, которые потребуются для дальнейшего изложения и понимания материалов.

1.1. АЛФАВИТ, ЦЕПОЧКА, КАТЕНАЦИЯ

Алфавит A – это некоторое непустое множество знаков, называемых символами.

Например, алфавит $A = \{0,1,2\}$, здесь 0, 1, 2 – символы алфавита A .

Цепочка – это некоторая последовательность символов алфавита A , возможно, повторяющихся. Например, цепочки 012, 001201, 1120 составлены из символов алфавита A . Для обозначения цепочки обычно используют малые буквы.

Число входящих в цепочку символов называют ее длиной. Так, у цепочки $x=012210$ длина, которая обозначается через $|x|$, равна 6.

Вводится аксиоматически так называемая пустая цепочка, обозначаемая через λ . Пустая цепочка это абстракция, она не содержит ни одного символа и имеет длину $|\lambda| = 0$.

Над цепочками можно проводить операцию катенации (соединения). Пусть цепочка $x = 012$ и цепочка $y = 210$, тогда катенация $xy = 012210$, а катенация $yx = 210012$.

Из определения цепочки λ следует справедливость следующих соотношений:

$$\begin{aligned}\lambda x &= x\lambda = x; \\ \lambda\lambda\lambda\dots\lambda &= \lambda.\end{aligned}$$

Повторяющуюся катенацию цепочки называют ее степенью:

$$\begin{aligned}x^0 &= \lambda \quad (\text{аксиома}); \\ x^1 &= x; \\ x^2 &= xx = xx^1; \\ &\dots\end{aligned}$$

$$x^n = \underbrace{xx \dots x}_{n\text{-раз}} = xx^{n-1}.$$

1.2. ГРАММАТИКА, СЕНТЕНЦИАЛЬНАЯ ФОРМА, ЯЗЫК

Формальной грамматикой $G[Z]$ называют непустое множество продукционных правил, а Z – начальным символом грамматики. Продукционное правило (продукция, правило вывода, правило подстановки) есть упорядоченная пара (U, x) , записываемая в виде

$$U ::= x,$$

где U – левая часть правила (в нашем случае символ), а x – непустая цепочка, образующая правую часть правила.

Грамматика задается на алфавите V , который называется словарем. Словарь V представлен двумя непересекающимися подмножествами V_T (словарь терминальных символов) и V_N (словарь нетерминальных символов). Терминальный символ может встречаться только в правой части правил, а нетерминальный – как в левой, так и правой частях правил грамматики.

Обычно в грамматике имена нетерминальных символов связывают с некоторым смыслом, для чего такие имена представляют многознаковыми последовательностями, составляющими название каких-либо понятий или их аббревиатур. Чтобы отличить их от цепочек символов, такие имена заключаются в угловые скобки и представляют они нетерминальные символы соответствующей грамматики. В качестве примера далее приведена грамматика $G[<\text{число}>]$, описывающая целые десятичные числа:

$$\begin{array}{lll} <\text{число}> & ::= & <\text{чс}> \\ <\text{чс}> & ::= & <\text{чс}><\text{цифра}> \\ <\text{чс}> & ::= & <\text{цифра}> \\ <\text{цифра}> & ::= & 0 \\ <\text{цифра}> & ::= & 1 \\ <\text{цифра}> & ::= & \begin{array}{c} \cdot \quad \cdot \quad \cdot \\ 9 \end{array} \end{array}$$

Здесь словарь грамматики представляет множество $V = \{ \langle \text{число} \rangle, \langle \text{чис} \rangle, \langle \text{цифра} \rangle, 0, 1, \dots, 9 \}$; терминальный словарь – множество $V_T = \{ 0, 1, \dots, 9 \}$ и словарь нетерминальных символов – множество $V_N = \{ \langle \text{число} \rangle, \langle \text{чис} \rangle, \langle \text{цифра} \rangle \}$.

Если правила содержат одинаковые левые части, то их объединяют в одно правило следующим образом:

$$U ::= x; U ::= y; U ::= z \longrightarrow U ::= x|y|z.$$

В символ $|$ вкладывается смысл логической связки ИЛИ. Такое представление грамматики называется нотацией Бэкуса – Наура.

С учетом данного обстоятельства модифицированная грамматика $G[\langle \text{число} \rangle]$ примет следующий вид:

$$\begin{aligned} \langle \text{число} \rangle & ::= \langle \text{чис} \rangle \\ \langle \text{чис} \rangle & ::= \langle \text{чис} \rangle \langle \text{цифра} \rangle | \langle \text{цифра} \rangle \\ \langle \text{цифра} \rangle & ::= 0|1|2|\dots|9 \end{aligned}$$

Теперь введем и определим смысл символа $\Rightarrow$. Пусть задана некоторая грамматика $G[Z]$ и в ней имеется правило $U ::= u$. Если имеется некоторая цепочка $x = aUb$, то из нее путем замены U на u по продукционному правилу $U ::= u$ получается новая цепочка $y = aub$. Такая замена записывается в виде

$$x \Rightarrow y.$$

В данном случае говорят, что цепочка x порождает цепочку y , или цепочка y выводима из цепочки x , или вывод цепочки y из цепочки x , или y редуцируется к x .

Если при выводе цепочки y из цепочки x подстановка выполняется только один раз, то это простой вывод или вывод длины 1. Если существует последовательность простых выводов и подстановка совершается n раз, то это вывод длины n , что записывается как

$$x \Rightarrow^+ y.$$

С понятием вывода очень тесно связано математическое понятие отношения. Действительно, если рассмотреть некоторое непустое множество цепочек, то на нем можно построить множество

пар цепочек, правая из которых в паре выводима из левой. Тогда в терминах отношений вывод представляется следующим образом:

- $\Rightarrow$ отношение простого вывода;
- $\Rightarrow+$ транзитивное замыкание отношения простого вывода;
- $\Rightarrow^*$ рефлексивное транзитивное замыкание отношения простого вывода.

Раскроем физический смысл отношения вывода. Пусть заданы две цепочки, a и b из некоторого конечного множества. Тогда отношение:

- $a \Rightarrow b$ представляет множество всех упорядоченных пар цепочек, для которых a всегда порождает b ;
- $a \Rightarrow+ b$ представляет множество всех упорядоченных пар цепочек, включающее отношение простого вывода, а также отношения вывода длины 2, 3, ..., n , ...;
- $a \Rightarrow^* b$ представляет объединение отношения $a \Rightarrow b$ и отношения $a = a$.

Сентенциальная форма. Пусть задана некоторая грамматика $G[Z]$, тогда цепочка y , выводимая из начального символа Z грамматики, называется сентенциальной формой, что записывается как

$$Z \Rightarrow^* y.$$

Предложение. Если сентенциальная форма y состоит только из терминальных символов, то она называется предложением.

Язык. Языком L грамматики $G[Z]$ называется множество всех предложений

$$L = \{y \mid y \text{ - предложение} \}.$$

Проиллюстрируем введенные понятия на примере грамматики $G[\langle \text{число} \rangle]$. В данной грамматике начальным символом является нетерминал $\langle \text{число} \rangle$.

- $\langle \text{число} \rangle \Rightarrow \langle \text{чс} \rangle$ – это простой вывод, где $x = \langle \text{число} \rangle$, $y = \langle \text{чс} \rangle$ и использовано правило вывода $\langle \text{число} \rangle ::= \langle \text{чс} \rangle$;

$\langle \text{число} \rangle \Rightarrow \langle \text{чс} \rangle \Rightarrow \langle \text{чс} \rangle$ – это вывод $\langle \text{число} \rangle \Rightarrow + 15$ и длина его $\Rightarrow \langle \text{чс} \rangle \langle \text{цифра} \rangle \Rightarrow$ равна 5. Здесь цепочка нетерминалов $15 \langle \text{цифра} \rangle \langle \text{цифра} \rangle \Rightarrow$ является предложением.
 $1 \langle \text{цифра} \rangle \Rightarrow 15$

Очевидно, что языком грамматики $G[\langle \text{число} \rangle]$ служит множество целых десятичных чисел.

Фраза. Это понятие – одно из ключевых в синтаксическом анализе.

Пусть задана грамматика $G[Z]$ и в ней для некоторого нетерминала U имеется выводимая из него подцепочка u :

$$U \Rightarrow +u .$$

Пусть имеется сентенциальная форма

$$x = aub ,$$

тогда в этой цепочке u есть фраза для нетерминала U , если при замене u на U образуется новая сентенциальная форма $y = a U b$, т.е.

$$Z \Rightarrow + a U b .$$

Фраза является **простой**, если вывод $U \Rightarrow u$ является простым, что эквивалентно наличию в грамматике правила $U ::= u$.

Например, рассмотрим для грамматики $G[\langle \text{число} \rangle]$ сентенциальную форму

$$x = \langle \text{чс} \rangle \langle \text{цифра} \rangle .$$

1. Проверим, является ли $\langle \text{чс} \rangle$ фразой для нетерминала $\langle \text{число} \rangle$. Выполним подстановку в соответствии с правилом $\langle \text{число} \rangle ::= \langle \text{чс} \rangle$, получим новую цепочку

$$y = \langle \text{число} \rangle \langle \text{цифра} \rangle .$$

Однако цепочка y не сентенциальная форма, следовательно, $\langle \text{чс} \rangle$ не фраза.

2. Проверим является ли $\langle \text{цифра} \rangle$ фразой для нетерминала $\langle \text{чс} \rangle$. Выполним подстановку в соответствии с правилом $\langle \text{чс} \rangle ::= \langle \text{цифра} \rangle$, получим новую цепочку

$$y = \langle \text{чс} \rangle \langle \text{чс} \rangle .$$

Здесь y также не сентенциальная форма, поэтому $\langle \text{цифра} \rangle$ также не фразой.

3. Проверим, вся ли цепочка $x = \langle \text{чс} \rangle \langle \text{цифра} \rangle$ является фразой для нетерминала $\langle \text{чс} \rangle$. Выполнив подстановку в соответствии с правилом $\langle \text{чс} \rangle ::= \langle \text{чс} \rangle \langle \text{цифра} \rangle$, получим новую цепочку

$$y = \langle \text{чс} \rangle .$$

Цепочка y – есть сентенциальная форма, т.е. $\langle \text{число} \rangle \Rightarrow \langle \text{чс} \rangle$, следовательно, вся цепочка x является фразой и причем простой.

1.3. ПОНЯТИЕ РАЗБОРА, СИНТАКСИЧЕСКИЕ ДЕРЕВЬЯ

Основная задача синтаксического анализа состоит в установлении факта принадлежности некоторой входной цепочки языку грамматики, а сам процесс установления принадлежности называется разбором.

Разбор может строиться по-разному: продвигаясь от начального символа, строится из него вывод цепочки x либо предпринимается попытка редуцировать цепочку x к начальному символу грамматики. В первом случае разбор называется нисходящим, а во втором – восходящим.

Наглядно разбор интерпретируется специальным графом, называемым синтаксическим деревом. На синтаксическом дереве вводится ряд понятий, смысл которых раскроем далее.

1. **Узлы** синтаксического дерева сопоставляются с нетерминальными и терминальными символами.

2. Узлы дерева связаны **ветвями**, отображающими вывод.

3. **Корнем** дерева называется узел, не имеющий входящих ветвей. Корень дерева соответствует начальному символу грамматики.

4. **Концевые узлы** – это узлы, не имеющие подчиненных узлов.

5. **Куст узла** – это множество подчиненных этому узлу других узлов.

6. **Поддерево** – это часть дерева, имеющая свой корень.

Проинтерпретируем данные понятия на примере дерева, соответствующего выводу $\langle \text{число} \rangle \Rightarrow + 15$ в грамматике $G[\langle \text{число} \rangle]$, что показано на рис.1:

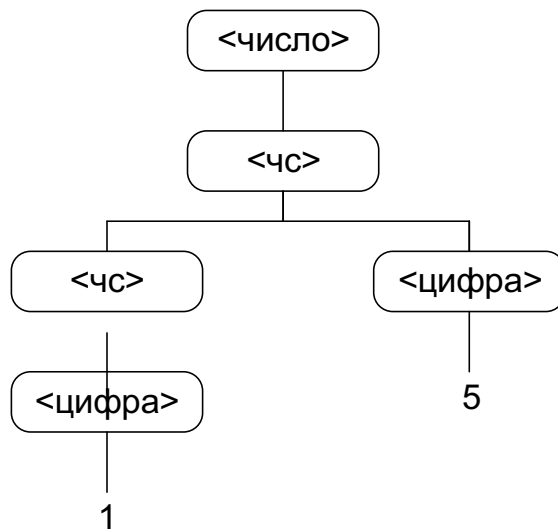


Рис.1. Пример синтаксического дерева для вывода $\langle \text{число} \rangle \Rightarrow + 15$

Здесь узел $\langle \text{число} \rangle$ есть корень дерева. Узлы 1 и 5 являются концевыми. В куст узла $\langle \text{чс} \rangle$ входят узлы $\langle \text{чс} \rangle$ и $\langle \text{цифра} \rangle$. Куст узла $\langle \text{число} \rangle$ образует единственный узел $\langle \text{чс} \rangle$.

Следующий пример показывает связь между выводом и синтаксическим деревом (рис. 2).

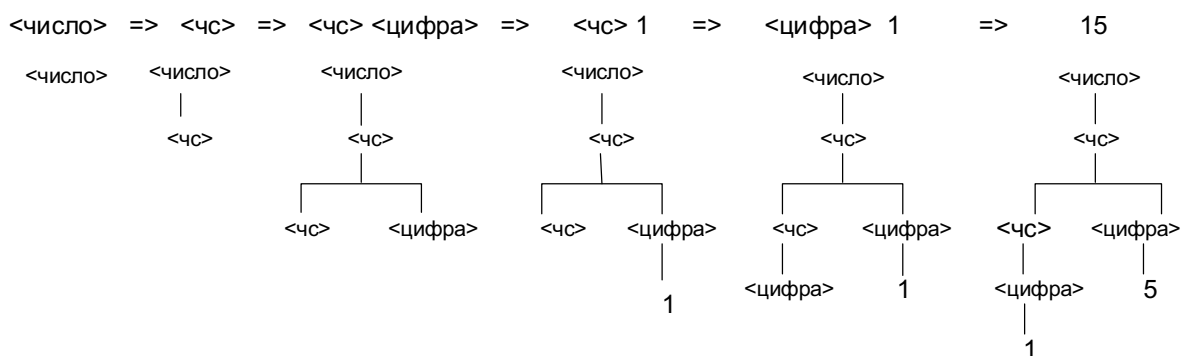


Рис.2. Связь вывода и синтаксического дерева

Из этого примера очевидно утверждение о том, что каждому выводу однозначно соответствует дерево, обратное чаще всего не

выполняется. Действительно, в выводе $\langle \text{число} \rangle \Rightarrow +15$ на промежуточном этапе из подцепочки $\langle \text{цифра} \rangle \langle \text{цифра} \rangle$ могут быть построены выводы:

$$\langle \text{цифра} \rangle \langle \text{цифра} \rangle \Rightarrow 1 \langle \text{цифра} \rangle \Rightarrow 15;$$

$$\langle \text{цифра} \rangle \langle \text{цифра} \rangle \Rightarrow \langle \text{цифра} \rangle 5 \Rightarrow 15.$$

Таким образом, одно и то же синтаксическое дерево может соответствовать более чем одному выводу.

Синтаксические деревья служат инструментом, позволяющим наглядно интерпретировать разбор. Действительно:

1) концевые узлы указывают на выводимую (редуцируемую) sentенциальную форму;

2) куст узла соответствует подцепочке, являющейся простой фразой для данного узла;

3) концевые узлы поддерева образуют фразу для корня этого поддерева.

Отсюда отсечение куста или его прорисовка на дереве эквивалентны построению простого вывода.

В контексте данного изложения укажем на ряд частных определений, не требующих детальных пояснений, которые используются в восходящем синтаксическом анализе.

Простой вывод

$$xUy \Rightarrow xuy$$

называется **каноническим**, если y — подцепочка, состоящая из терминалов.

Вывод

$$V \Rightarrow^+ W$$

называется **каноническим**, если он состоит из последовательности простых канонических выводов.

Определение. **Сентенциальная форма**, имеющая канонический вывод, называется **канонической**.

Определение. Самая левая простая фраза сентенциальной формы называется **основой**.

Обычно для обозначения канонического вывода вместо символа $\Rightarrow$ используется символ $\Rightarrow^>$.

1.4. КЛАССИФИКАЦИЯ ЯЗЫКОВ

В 1959 г. американский ученый Н. Хомский предложил классифицировать языки по типу правил подстановки порождающей грамматики. Порождающая грамматика определяется четверкой:

$$(V, V_T, P, Z),$$

где V – алфавит; V_T – словарь терминальных символов; P – множество продукционных правил; Z – начальный символ.

Тогда классификация выглядит следующим образом:

0-грамматики имеют правила

$$u ::= +v,$$

где $u \in V^+$ и $v \in V^*$.

0-грамматики называют еще грамматиками фразовой структуры. Они моделируют естественные языки.

1-грамматики имеют правила вывода вида

$$xUy ::= xuy,$$

где $U \in \mathbb{N}V_T$; $u \in V^+$; $x \in V^*$; $y \in V^*$.

Грамматики данного класса называются контекстно-чувствительными, поскольку подстановка u вместо U может быть выполнена только в контексте x и y .

2-грамматики имеют правила вывода вида

$$U ::= u,$$

где $U \in \mathbb{N}V_T$; $u \in V^+$.

Эти грамматики называют контекстно свободными (КС-грамматиками). Чаще всего именно этот тип грамматик моделирует языки программирования.

3-грамматики имеют правила вывода вида

$$U ::= T \text{ или } U ::= WT,$$

где $T \in V_T$, $U \in \mathbb{N}V_T$, $W \in \mathbb{N}V_T$.

Грамматики данного класса называют А-грамматиками или автоматными. Они порождают так называемые регулярные языки, разбор предложений которых может быть выполнен конечным автоматом. Подробно все эти вопросы будут рассмотрены в следующих разделах.

2. КОНЕЧНЫЕ РАСПОЗНАВАТЕЛИ

Настоящий раздел знакомит читателя с понятием конечного распознавателя, раскрывает его связь с автоматными грамматиками и регулярными выражениями.

2.1. ДЕТЕРМИНИРОВАННЫЙ КОНЕЧНЫЙ РАСПОЗНАВАТЕЛЬ

Детерминированный конечный распознаватель (ДКР) – это пятерка:

$$(K, V, M, S, Z),$$

где K – алфавит состояний; V – входной алфавит; M – функция переходов (представляет отображение $K^*V \rightarrow K$); S – начальное состояние; Z – множество заключительных состояний.

Для понимания физики работы ДКР лучше всего представлять его в виде устройства, имеющего конечное число *внутренних состояний* K . Под воздействием цепочек символов, состоящих из символов входного алфавита, ДКР изменяет текущее состояние, реализуя **функцию переходов** M . В нашем случае ДКР может распознать некоторое множество цепочек, образующее **входной язык**, а остальные отвергнуть, для этого часть состояний $Z \subset K$ отмечается в качестве *допускающих*, а остальные – *отвергающих*. Если распознаватель после прочтения последней буквы цепочки закончил работу в заключительном (допускающем) состоянии, то цепочка допускается (распознана), в противном случае – не допускается (отвергнута). Работу ДКР начинает всегда с некоторого состояния S , называемого **начальным**.

Таким образом, в ДКР функция выходов реализована на множестве состояний и может иметь два значения: либо 0 (false), либо 1 (true). Если ДКР закончил сканирование входной цепочки в недопускающем состоянии, то значение выхода есть 0, если в допускающем состоянии, то значение выхода принимает значение 1. Таким образом, распознаватель представляет собой типичный автомат Мура с одним бинарным выходом.

Очень удобно с точки зрения наглядности представить ДКР диаграммой состояний. **Диаграмма состояний** – это ориентированный граф, вершины которого сопоставлены состояниям распознавателя, а дуги размечены символами входного алфавита.

Далее представлена диаграмма состояний конечного распознавателя, который допускает цепочки символов входного алфавита $\{0,1\}$, состоящие из любого числа пар одинаковых символов (рис. 3).

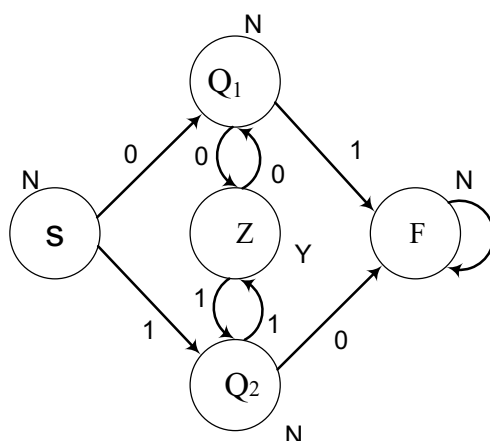


Рис.3. Диаграмма состояний ДКР, распознающего последовательности из пар 00 или 11

Здесь S – начальное состояние; Z – заключительное состояние; Q_1 и Q_2 – состояния автомата; F – состояние ошибки.

Функция переходов M определяет последующее состояние КА Q_1^{t+1} через текущее состояние Q_m^t и входной символ C^t , т.е.

$$Q_1^{t+1} = M(Q_1^t, C^t).$$

Обычно надстрочные символы дискретного времени опускают, подразумевая их.

Так, для рассмотренного ранее примера ДКР функция переходов, представленная всеми своими частными значениями, имеет следующий вид:

$$\begin{aligned} M(S,0)=Q_1; \quad M(Q_1,0)=Z; \quad M(Q_2,0)=F; \quad M(Z,0)=Q_1; \quad M(F,0)=F; \\ M(S,1)=Q_2; \quad M(Q_1,1)=F; \quad M(Q_2,1)=Z; \quad M(Z,1)=Q_2; \quad M(F,1)=F. \end{aligned}$$

Обычно функцию переходов представляют в виде таблицы переходов. В ней столбцы поименованы буквами входного алфавита V , строки – буквами алфавита состояний K , а в клетку на пересечении строки и столбца записывается значение частичной функции переходов $M(Q, C)$. В таблицу также вводится еще один дополнительный столбец (столбец меток), соответствующий значению функции выходов, которое привязывается к конкретному состоянию. Символы меток Y и N отмечают допускающие и отвергающие состояния соответственно. По таблице переходов можно полностью восстановить поведение распознавателя при разборе любой входной цепочки. Таким образом, диаграмма состояний и таблица переходов представляют две нотации описания и представления конечного распознавателя.

Приведем таблицу переходов рассмотренного ранее примера детерминированного конечного распознавателя.

$K \backslash V$	0	1	Метка
S	Q_1	Q_2	N
Q_1	Z	F	N
Q_2	F	Z	N
Z	Q_1	Q_2	Y
F	F	F	N

Последовательность смены состояний данного распознавателя при обработке входной цепочки 00110011 представлена в виде следующей таблицы:

Входная цепочка		0		0		1		1		0		0		1		1	
Смена состояний	S	$\rightarrow$	Q_1	$\rightarrow$	Z	$\rightarrow$	Q_2	$\rightarrow$	Z	$\rightarrow$	Q_1	$\rightarrow$	Z	$\rightarrow$	Q_2	$\rightarrow$	Z

Данная цепочка допускается распознавателем, поскольку последнее состояние Z является допускающим. Такое представление зачастую называют линейной разверткой распознавателя.

Теперь представим разбор цепочки несколько по-иному (рис.4).

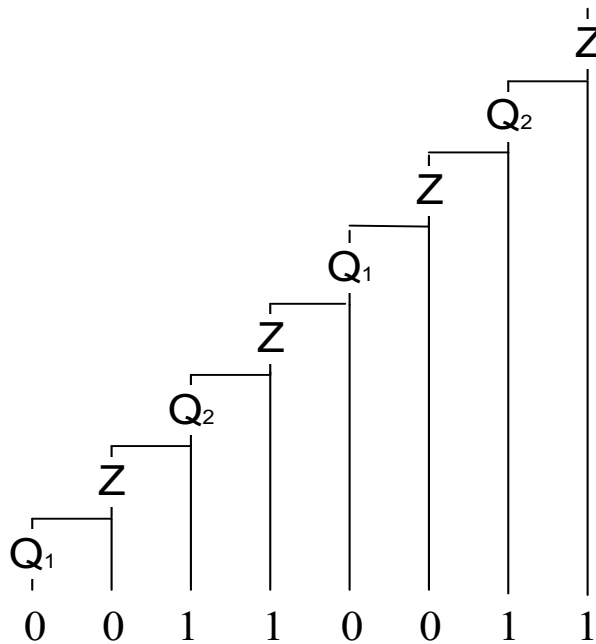


Рис.4. Дерево разбора цепочки 00110011

Данное представление – это дерево разбора, а его построение можно интерпретировать как разбор цепочки по правилам некоторой грамматики G . По данному синтаксическому дереву нетрудно восстановить вывод и грамматику:

$$\begin{aligned} Z &::= Q_2 1 | Q_1 0 \\ Q_1 &::= Z 1 | 1 \\ Q_2 &::= Z 0 | 0 \end{aligned}$$

Словарь грамматики образуют нетерминалы Z , Q_1 , Q_2 и терминалы 1 и 0.

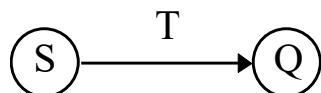
Приведенное в качестве примера построение грамматики по разбору цепочки наглядно иллюстрирует связь грамматик и распознавателей. В то же время при построении конечных распознавателей зачастую решают задачу обратную, переходят от грамматики к распознавателю.

Построение распознавателя по грамматике может быть выполнено по определенным правилам.

- 1) Отождествить нетерминалы с состояниями распознавателя, терминалы – со входными символами.
- 2) Ввести дополнительно начальное состояние S .
- 3) Для всех правил вида

$$Q ::= T,$$

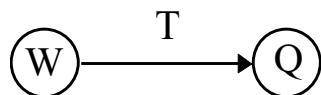
где $Q \in V_N$, $T \in V_T$, выполнить построение следующего фрагмента диаграммы состояний:



4) Для всех правил вида

$$Q ::= WT,$$

где $Q \in V_N$, $W \in V_N$, $T \in V_T$, выполнить построение фрагмента диаграммы состояний вида:



5) Разметить в диаграмме состояний отвергающие и допускающие состояния.

Рассмотрим пример построения конечного распознавателя по грамматике. Пусть задана следующая грамматика $G[Z]$:

$$\begin{aligned} Z &::= Q_1 0 | Q_2 1 \\ Q_1 &::= Z 0 | 0 \\ Q_2 &::= Z 1 | 1 \end{aligned}$$

и по ней требуется построить конечный распознаватель.

В соответствии с алгоритмом поставим в соответствие нетерминалам грамматики Z , Q_1 , Q_2 состояния, а терминалам 0 и 1 – символы входного алфавита распознавателя. Допускающим состоянием в распознавателе следует сделать состояние Z . После выполнения данных действий диаграмма состояний распознавателя для грамматике $G[Z]$ выглядит следующим образом (рис.5):

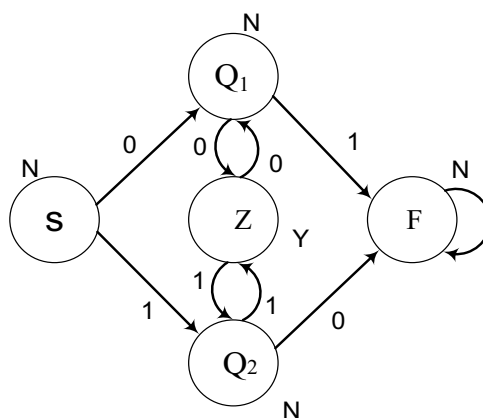


Рис.5. Диаграмма состояний распознавателя грамматики $G[Z]$

Таким образом, между грамматиками и распознавателями существует тесная глубинная связь, эта связь выражается через общий язык – грамматика порождает язык, а распознаватель его распознает. Но эта связь не всегда так проста, как в случае приведенного примера автоматных грамматик.

2.2. ПРАВОСТОРОННИЕ АВТОМАТНЫЕ ГРАММАТИКИ

В разд. 2.1 рассуждения были неявно построены относительно левосторонних автоматных грамматик. В то же время существуют и правосторонние автоматные грамматики, которые обладают рядом замечательных свойств. Они характеризуются тем, что имеют правила вида

$$\begin{array}{ccccc}
 & Z & ::= & T & Q , \\
 \nearrow & & & \nearrow & \nwarrow \\
 \text{нетерминал} & & & \text{терминал} & \text{нетерминал}
 \end{array}$$

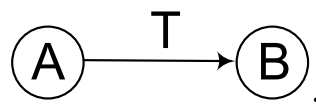
у которых нетерминал в левой части правила занимает правую позицию. Грамматики такого вида называются правосторонними автоматными грамматиками.

2.3. ПЕРЕХОД ОТ КОНЕЧНОГО РАСПОЗНАВАТЕЛЯ К ПРАВОСТОРОННИМ ГРАММАТИКАМ

Пусть задан конечный распознаватель, по которому требуется построить правостороннюю автоматную грамматику.

Алгоритм:

- 1) терминальным словарем грамматики сделать множества входных символов алфавита распознавателя;
- 2) нетерминальным словарем сделать множество состояний автомата, при этом начальным символом грамматики считать начальное состояние распознавателя;
- 3) если в автомате имеется переход вида:



то в грамматике ввести правило вида

$$A ::= TB;$$

- 4) если некоторое состояние A в автомате является конечным, то для него действует правило $A ::= \lambda$, где λ – пустая цепочка.

Проиллюстрируем работу алгоритма на конкретном примере распознавателя, который представлен следующей диаграммой состояний (рис. 6):

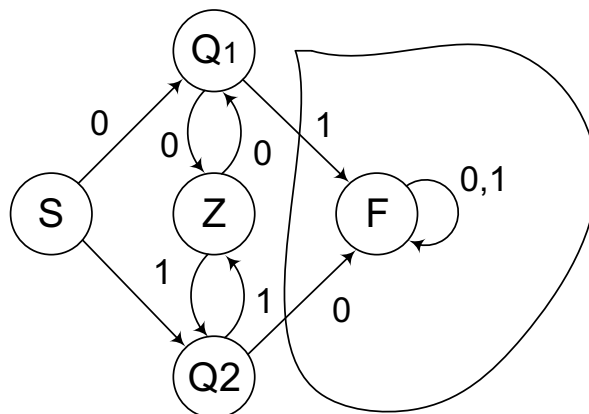


Рис. 6. Диаграмма состояний распознавателя, построенного по правосторонней автоматной грамматике

После проведения построения, предписанного алгоритмом, получим грамматику (представлена слева), в которой содержатся

правила с одинаковыми левыми частями. Объединяя такие правила с помощью связки ИЛИ в одно правило, получим модифицированную нормальную форму Бэкуса – Наура грамматики (представлена в таблице справа).

Грамматика		Модифицированная грамматика
$S ::= 0Q_1$ $S ::= 1Q_2$ $Q_1 ::= 0Z$ $Q_2 ::= 1Z$ $Z ::= 0Q_1$ $Z ::= 1Q_2$ $Z ::= \lambda$		$S ::= 0Q_1 1Q_2$ $Q_1 ::= 0Z$ $Q_2 ::= 1Z$ $Z ::= 0Q_1 1Q_2 \lambda$

Синтаксическое дерево разбора для цепочки $x=0011\lambda$ в полученной грамматике имеет вид (рис. 7):

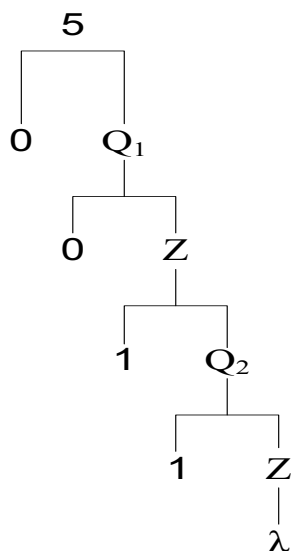


Рис.7. Синтаксическое дерево разбора цепочки $x=0011\lambda$

Для понимания процесса построения правосторонней грамматики по распознавателю необходимо иметь в виду определенную интерпретацию распознавания некоторой цепочки.

1. Для начального символа S . Цепочка допускается распознавателем, начавшим работу в состоянии S .

2. Для правила вида $A ::= TB$. Цепочка, допускаемая распознавателем, начавшим работу в состоянии A , представляет собой цепочку, начинающую с терминала T и приписанную к нему справа цепочку, допускаемую распознавателем, начавшим работу с состояния B .

3. Для правила $A ::= \lambda$. Цепочка, допускаемая распознавателем, начавшим свою работу в состоянии A , может быть пустой.

2.4. КВАЗИАВТОМАТНЫЕ ГРАММАТИКИ И ИХ ПРИВЕДЕНИЕ К ПРАВОСТОРОННИМ АВТОМАТНЫМ ГРАММАТИКАМ

Рассмотрим еще один класс правосторонних грамматик. Правила в этих грамматиках имеют вид $A ::= wB$, где w – цепочка из терминальных символов, а A и B – нетерминалы. Такие грамматики не являются автоматными, но их достаточно просто можно преобразовать к автоматному виду $A ::= TB$.

Поясним данную процедуру на конкретном примере. Пусть задана следующая грамматика:

№ правила	Правило
1	$S ::= aA$
2	$S ::= bc$
3	$S ::= A$
4	$A ::= abS$
5	$A ::= cA$
6	$A ::= \lambda$

В данной грамматике нетерминалы обозначаются прописными буквами, а терминалы – строчными.

Теперь проведем в грамматике некоторые преобразования, которые не изменяют порождаемый ею язык.

Преобразование 1. Рассмотрим 4-е правило и введем дополнительный нетерминал $\langle bS \rangle$. Построим для него новое правило

$\langle bS \rangle ::= bS$. Теперь 2-е правило исходной грамматики представляется в виде двух правил:

Исходное правило	Результат преобразования
$A ::= abS$	$A ::= a\langle bS \rangle$ $\langle bS \rangle ::= bS$

Преобразование 2. Изменим 2-е правило. В данном правиле левая часть представляет катенацию двух терминалов, поэтому оно не является автоматным. Путем введения новых нетерминалов $\langle \text{слямба} \rangle$ и $\langle \text{лямбда} \rangle$ сводим его к трем автоматным правилам:

№ шага	Преобразования	Примечание
1	$S ::= b\langle \text{слямба} \rangle$	Введен новый нетерминал $\langle \text{слямба} \rangle$
2	$S ::= b\langle \text{слямба} \rangle$ $\langle \text{слямба} \rangle ::= c\langle \text{лямбда} \rangle$ $\langle \text{лямбда} \rangle ::= \lambda$	Для нетерминалов $\langle \text{слямба} \rangle$ и $\langle \text{лямбда} \rangle$ ввели два новых правила

Преобразование 3. Рассмотрим 3-е правило $S ::= A$. Поскольку при преобразовании 1 образовалось новое правило $A ::= a\langle bS \rangle$, то 3-е правило грамматики заменим на него:

Исходное правило	Замена правила
$S ::= A$	$S ::= a\langle bS \rangle$

Преобразование 4. Сведем все эти правила вместе, образовав новую грамматику:

№ правила	Исходная грамматика	Результат преобразования	Новая грамматика
1	$S ::= aA$		$S ::= aA$
2	$S ::= bc$	2а. $S ::= b\langle c\text{ лямбда} \rangle$ 2б. $\langle \text{лямбда} \rangle ::= \lambda$	$S ::= b\langle c\text{ лямбда} \rangle$ $\langle \text{лямбда} \rangle ::= \lambda$

3	$S ::= A$	$S ::= a \langle bS \rangle,$ $S ::= cA,$ $S ::= \lambda$	$S ::= a \langle bS \rangle$ $S ::= cA$ $S ::= \lambda$
4	$A ::= abS$	4a. $A ::= a \langle bS \rangle$ 4b. $\langle bS \rangle ::= bS$	$A ::= a \langle bS \rangle$ $\langle bS \rangle ::= bS$
5	$A ::= cA$		$A ::= cA$
6	$A ::= \lambda$		$A ::= \lambda$

Выполним процедуру объединения правых частей правил, у которых левые части одинаковые, в одно правило с помощью логической связки ИЛИ. После данного преобразования получим новую грамматику вида:

$$\begin{aligned}
 S &::= aA \mid a \langle bS \rangle \mid cA \mid b \langle \text{сямбда} \rangle \mid \lambda \\
 \langle \text{сямбда} \rangle &::= c \langle \text{лямбда} \rangle \\
 \langle \text{лямбда} \rangle &::= \lambda \\
 \langle bS \rangle &::= bS
 \end{aligned}$$

Соответствующий данной грамматике конечный распознаватель представлен далее (рис. 8):

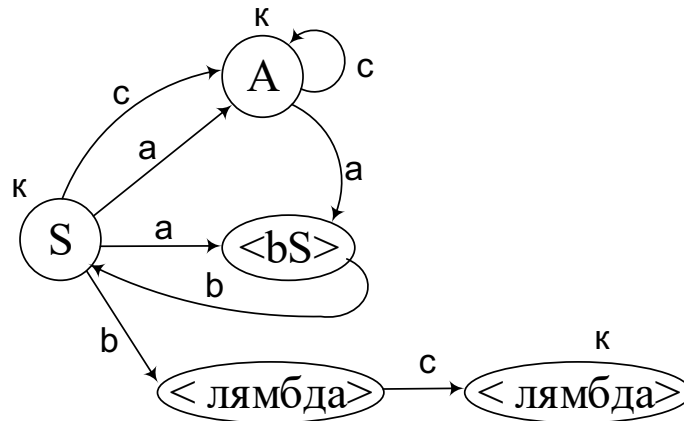


Рис. 8. Диаграмма состояний распознавателя

Конечный распознаватель, построенный по данной грамматике, является недетерминированным, так как из состояния S по входному символу a переход осуществляется сразу в два состояния A и $\langle bS \rangle$.

Примечание. Недетерминированные конечные распознаватели будут подробно рассмотрены далее.

Сформулируем теперь общую процедуру преобразования правосторонних квазиавтоматных грамматик к автоматному виду.

Если в грамматике имеются продукционные правила вида

1. $A ::= W$
2. $A ::= WQ$
3. $A ::= B$
4. $A ::= A ,$

то процедура их преобразования к автоматному виду сводится к следующим действиям.

Для первого правила

$$A ::= W \quad \left\{ \begin{array}{l} \rightarrow A ::= W \langle \text{лямбда} \rangle \\ \langle \text{лямбда} \rangle ::= \lambda . \end{array} \right.$$

Для второго правила

$$\begin{aligned} A ::= a_1 a_2 \dots a_n Q &\rightarrow A ::= a_1 \langle a_2 \dots a_n Q \rangle \\ &\langle a_2 \dots a_n Q \rangle ::= a_2 \langle a_3 \dots a_n Q \rangle \\ &\text{-----} \\ &\langle a_n Q \rangle ::= a_n Q . \end{aligned}$$

Для третьего правила: если имеется правило вида $A ::= B$ и существует правило $B ::= Y$, то ввести правило $A ::= Y$.

Для четвертого правила: если имеются правила вида $A ::= A$, то они являются лишними и их нужно удалить.

Таким образом, используя предложенные правила, можно свести квазиавтоматную грамматику к автоматной и далее по известному алгоритму синтезировать конечный распознаватель.

2.5. МИНИМИЗАЦИЯ ДЕТЕРМИНИРОВАННЫХ КОНЕЧНЫХ РАСПОЗНАВАТЕЛЕЙ

В математической логике одну и ту же булеву функцию может представлять множество дизъюнктивных нормальных форм,

при этом как минимум одна из них является минимальной. Ее поиск и есть цель задачи минимизации булевых функций. Данный взгляд по аналогии возможно распространить и на конечные распознаватели, когда одно и то же множество входных цепочек может быть распознано некоторым множеством конечных распознавателей. В этом смысле все они эквивалентные, поскольку распознают один и тот же язык, но отличаются друг от друга числом состояний и функциями переходов. Среди таких распознавателей существует по крайней мере один с минимальным числом состояний. Его поиск и составляет цель задачи минимизации конечных распознавателей. В этой связи дальнейшее изложение посвящено построению минимального по числу состояний распознавателя из некоторого исходного представления.

Пусть задан некоторый детерминированный конечный распознаватель. Его таблица переходов имеет следующий вид:

	0	1	Y/N
S	U_1	U_2	N
U_1	B_1	F_1	N
U_2	F_1	B_1	N
B_1	U_3	U_4	Y
F_2	F_2	F_2	N
U_3	B_2	F_1	N
U_4	F_1	B_2	N
B_2	U_3	U_4	Y
F_1	F_1	F_1	N
Z	—	B_3	Y
B_3	B_4	U_1	Y
B_4	U_2	Z	Y

Диаграмма состояний детерминированного конечного распознавателя показана на рис. 9:

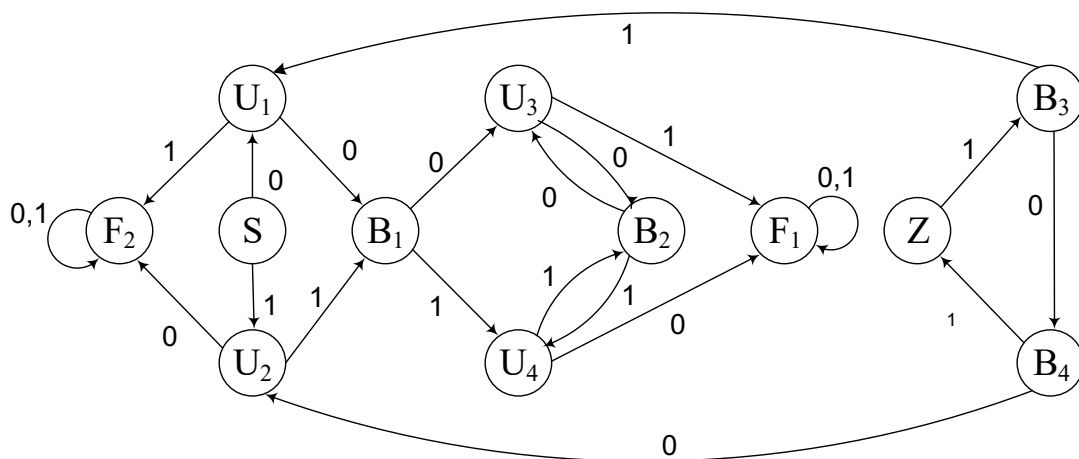


Рис.9. Диаграмма состояний распознавателя

В распознавателе S является начальным состоянием, B_1, B_2, B_3, B_4, Z – допускающими состояниями, а остальные состояния – отвергающими. Нетрудно убедиться, что детерминированный конечный распознаватель допускает цепочки, состоящие из любого количества пар 00 и 11.

Проинтерпретируем некоторые понятия на данном примере.

Недостижимые состояния. Среди состояний распознавателя могут присутствовать такие состояния, которые недостижимы из начального состояния ни для какой входной цепочки. Соответствующие строки в таблице переходов, в также соответствующие им вершины и связи в диаграмме состояний можно без ущерба удалить, поскольку они никогда не участвуют в процессе распознавания. В примере строки недостижимых состояний выделены двойной рамкой.

Тривиальный алгоритм удаления недостижимых состояний заключается в построении новой таблицы переходов из старой по следующему алгоритму.

На первом шаге создается пустая таблица переходов и именуется первая строка начальным состоянием S :

	0	1	Y/N
S			

Далее из старой таблицы переходов в новую таблицу переносятся состояния, соответствующие значениям частичной функции переходов для состояния S :

	0	1	Y/N
S	U_1	U_1	

Для состояний U_1 и U_2 заводятся новые строки:

	0	1	Y/N
S	U_1	U_2	
U_1			
U_2			

В ячейки строк U_1 и U_2 в соответствии с функцией переходов вписываются новые состояния:

	0	1	Y/N
S	U_1	U_2	
U_1	B_1	F_2	
U_2	F_2	B_1	

Процесс продолжается до тех пор, пока не прекратится дописывание строк новых состояний. Теперь в полученной новой таблице переходов будут отсутствовать все недостижимые состояния.

Для данного примера таблица переходов без недостижимых состояний примет следующий вид:

	0	1	Y/N
S	U_1	U_2	N
U_1	B_1	F_2	N
U_2	F_2	B_1	N
B_1	U_3	U_4	Y
F_2	F_2	F_2	N
U_3	B_2	F_1	N
U_4	F_1	B_2	N

B_2	U_3	U_4	Y
F_1	F_1	F_1	N

Соответственно недостижимые состояния уйдут и из диаграммы состояний распознавателя, которая будет иметь следующий вид (рис.10):

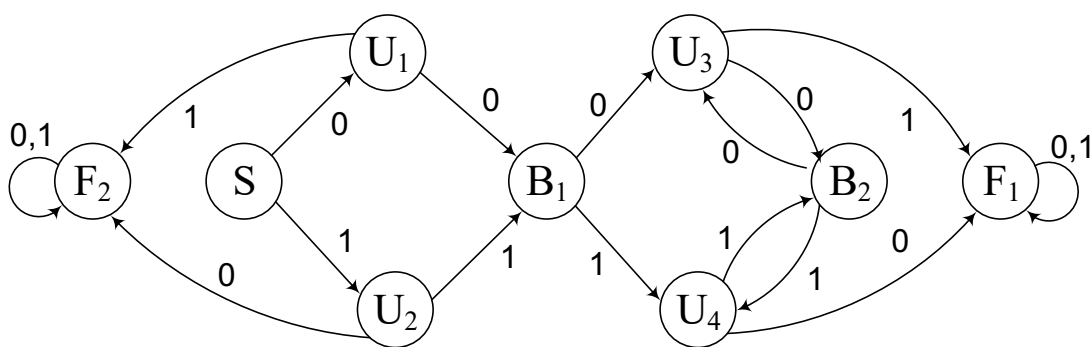


Рис.10 Диаграмма состояний распознавателя без недостижимых состояний

После выполнения удаления недостижимых состояний необходимо перейти непосредственно к поиску эквивалентных состояний.

Эквивалентность состояний. Два состояния Q и L являются эквивалентными, если:

- 1) оба они допускающие или отвергающие (условие подобия);
- 2) любая цепочка t переводит их в эквивалентные состояния P и R (условие преемственности) (рис.11):

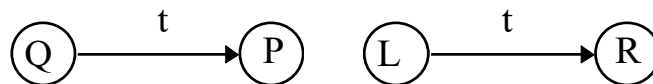


Рис.11. Преемственность состояний

Таким образом, два состояния эквивалентны, если не существует различающая их цепочка.

Отметим, что эквивалентность состояний является в математическом смысле отношением эквивалентности на множестве состояний распознавателя. Отношение рефлексивно (каждое состояние эквивалентно само себе), симметрично (если Q эквивалентно

L , то L эквивалентно Q), транзитивно (если Q эквивалентно L , а L эквивалентно P , то Q эквивалентно P).

Эквивалентные состояния образуют непересекающиеся классы эквивалентностей, каждый из которых можно представить одним эквивалентным состоянием.

Для рассматриваемого примера проверим эквивалентность состояний B_1 и B_2 . Для этих состояний выполняется условие подобия – оба они допускающие. Проверку преemannности удобно проводить путем построения таблицы эквивалентности. В ней столбцы соответствуют входным символам, а строки – проверяемым парам:

Шаг 1. Выделим случайным образом пару состояний B_1 и B_2 , которую проверим на эквивалентность:

Пары	0	1	Примечание
B_1, B_2			Условие подобия выполняется

Шаг 2. В клетки таблицы в соответствии с функцией переходов впишем пары преемников для входных символов 0 и 1 соответственно.

	0	1	
B_1, B_2	U_3, U_3	U_4, U_4	Преемники в парах подобны

Пары преемников эквивалентны в силу свойства рефлексивности. Отсюда состояния B_1 и B_2 также эквивалентны. Если эквивалентность по преемникам установить не удалось, то необходимо продолжить дописывание строк для пар преемников и проверить их также на эквивалентность и т.д.

Проверяя подобным образом все пары состояний, нетрудно убедиться, что классами эквивалентных состояний будут следующие:

$$B=\{B_1, B_2\}, F=\{F_1, F_2\}, U_{13}=\{U_1, U_3\}, U_{24}=\{U_2, U_4\}.$$

Теперь выполним следующее построение. Заменим в таблице переходов все состояния на имена их эквивалентных классов, после чего получим следующую таблицу переходов:

	0	1	Y/N
S	U_{13}	U_{24}	N
U_{13}	B	F	N
U_{24}	F	B	N
B	U_{13}	U_{24}	Y
F	F	F	N
U_{13}	B	F	N
U_{24}	F	B	N
B	U_{13}	U_{24}	Y
F	F	F	N

В данной таблице имеются одинаковые строки и их число определяется числом элементов, входящих в классы эквивалентностей. Перепишем вновь таблицу переходов, оставив только по одной строке – представителю класса, после чего получим новую таблицу переходов, которая представляет минимальный распознаватель:

	0	1	
S	U_{13}	U_{24}	N
U_{13}	B	F	N
U_{24}	F	B	N
B	U_{13}	U_{24}	Y
F	F	F	N

Иллюстрация фрагмента процесса минимизации конечного распознавателя диаграммами состояний представлена на рис.12.

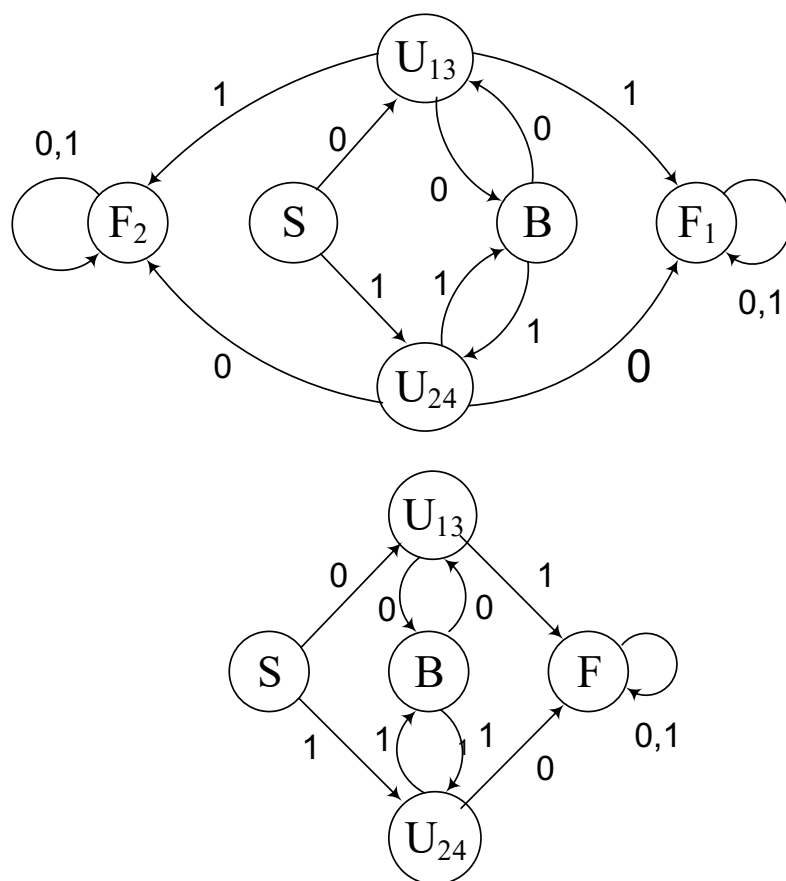


Рис.12. Графическое представление процесса минимизации

Здесь показан процесс стягивания эквивалентных состояний F_1 и F_2 (верхняя диаграмма состояний) в эквивалентное состояние F (нижняя диаграмма состояний). Состояния U_{13} , U_{24} и B есть имена эквивалентных классов состояний, они уже представлены на диаграмме состояний.

Отметим, что таблицы эквивалентностей позволяют попутно с проверяемой на эквивалентность парой состояний выявить эквивалентность и других пар, начав проверку с данной пары. Тем не менее полной уверенности в том, что проверены на эквивалентность все пары состояний и выявлены все эквивалентные состояния, нет. Поэтому в худшем случае необходимо выполнить перебор всех пар.

Данного недостатка лишен более эффективный метод поиска и объединения эквивалентных состояний, называемый зачастую методом разбиения на блоки. Его суть заключается в следующем.

1. Сначала состояния автомата разбиваются на блоки, один из которых включает допускающие состояния, а другой – отвергающие, т.е. разбиение происходит по условию подобия. В нашем случае разбиение имеет вид

$$P_0 = (\underbrace{\{S, U_1, U_2, F_2, U_3, U_4, F_1\}}_{\text{1-й блок}}, \underbrace{\{B_1, B_2\}}_{\text{2-й блок}}).$$

Очевидно, что ни одно из состояний 1-го блока не эквивалентно ни одному из состояний 2-го блока, поскольку не выполняется условие подобия. Поэтому эквивалентные состояния могут находиться только в пределах одного блока (условие подобия).

Далее правило разбиения сводится к следующему. При образовании нового блока в разбиении P_i в него надо включать только те состояния, которые имеют преемников в пределах одного блока предыдущего разбиения P_{i-1} .

Покажем действия по разбиению блоков с учетом данного правила в случае нашего примера распознавателя

2. Разобьем P_0 относительно входного символа 0.

По таблице переходов в 1-м блоке состояниями-преемниками для $\{S, U_2, F_2, U_4, F_1\}$ являются состояния 1-го блока, а для $\{U_1, U_3\}$ состояния-преемники находятся во втором блоке. Следовательно, U_1 и U_3 не могут быть эквивалентны ни одному из состояний: S, U_2, F_2, U_4, F_1 .

Таким образом, 1-й-блок в P_0 разбивается на два. Аналогичную процедуру разбиения попытаемся провести над 2-м блоком разбиения P_0 . Однако преемник состояний 2-го блока один, это состояние U_3 , оно находится в 1-м, поэтому 2-блок не распадается. С учетом сказанного получим новое разбиение P_1 :

$$P_1 = (\underbrace{\{S, U_2, F_2, U_4, F_1\}}_{\text{1-й блок}}, \underbrace{\{U_1, U_3\}}_{\text{2-й блок}}, \underbrace{\{B_1, B_2\}}_{\text{3-й блок}}).$$

3. Таким же образом проведем разбиение над P_1 относительно символа 1, после чего получим разбиение P_2 , которое имеет следующий вид:

$$P_2 = (\{S, F_2, F_1\}, \{U_2, U_4\}, \{U_1, U_3\}, \{B_1, B_2\}).$$

4. Проведем новое разбиение P_2 относительно символа 0:

$$P_3 = (\{S\}, \{F_2, F_1\}, \{U_2, U_4\}, \{U_1, U_3\}, \{B_1, B_2\}).$$

Дальнейшего расщепления блоков не происходит, следовательно, блоки содержат эквивалентные состояния.

Теперь поименуем каждый из блоков следующим образом:

$$\begin{aligned} \{S\} &\rightarrow S \\ \{F_2, F_1\} &\rightarrow F \\ \{U_2, U_4\} &\rightarrow U_{24} \\ \{U_1, U_3\} &\rightarrow U_{13} \\ \{B_1, B_2\} &\rightarrow B \end{aligned}$$

Далее произведем замену всех эквивалентных состояний на имена блоков эквивалентностей и удалим повторяющиеся строки, после чего получим следующую таблицу переходов:

	0	1
S	U_{13}	U_{24}
U_{13}	Z	F
U_{24}	F	Z
F	F	F
Z	U_{13}	U_{24}

Данная таблица переходов аналогична таблице переходов, полученной ранее методом таблиц эквивалентностей. Заметим, что исходный конечный распознаватель имеет 12 состояний, в то время как минимизированный – 5 состояний.

Нетрудно заметить, что метод разбиения на блоки является модификацией метода таблиц эквивалентностей, отличительная особенность которого состоит в том, что проверка осуществляется сразу всех состояний распознавателя.

Положительный эффект от минимизации замечен при сравнении диаграмм состояний исходного и минимизированного распознавателей. Отсюда следует вывод: этапом минимизации пренебрегать нельзя.

2.6. НЕДЕТЕРМИНИРОВАННЫЙ КОНЕЧНЫЙ РАСПОЗНАВАТЕЛЬ

Недетерминированный конечный распознаватель (НКР) – это пятерка

$$(K, V, M, S, Z),$$

где K – алфавит состояний, V – входной алфавит, M – функция переходов, представляющая отображение $K \times V \rightarrow$ в подмножество K ; S – множество начальных состояний ($S \subset K$); Z – множество заключительных состояний.

Обратим внимание на принципиально различные определения в ДКР и НКР таких атрибутов, как начальные состояния (одно в ДКР и несколько в НКР) и функции переходов.

Как правило, НКР порождаются автоматными грамматиками, которые содержат правила вида

$$U ::= WT \text{ и } L ::= WT,$$

что соответствует следующему фрагменту диаграммы состояний (рис. 13):

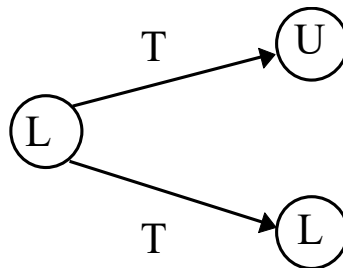


Рис. 13. Фрагмент диаграммы НКР

Частичная функция переходов недетерминированного распознавателя для данного случая имеет вид

$$M(W, T) = \{U, L\}.$$

Цепочка t допускается НКР, если в диаграмме состояний найдется для нее путь хотя бы из одного некоторого состояния $P \in S$ в хотя бы в одно состояние $Q \in Z$.

Пусть задана регулярная грамматика $G[Z]$:

$$\begin{aligned}
Z &::= U1|V0|Z0|Z1 \\
U &::= Q1|1 \\
V &::= Q0|0 \\
Q &::= Q0|Q1|0|1
\end{aligned}$$

Произведем переход от грамматики к недетерминированному конечному распознавателю и построим его диаграмму состояний. Выделим атрибуты распознавателя.

1. Алфавит состояний $K = \{S, Q, V, U, Z\}$.
2. Входной алфавит $V = \{0, 1\}$.
3. Множество заключительных состояний $\{Z\}$.
4. Множество начальных состояний $\{S\}$.
5. Функция переходов, заданная перечислением:

$M(S,0)=\{V,Q\}$	$M(V,0)=\{Z\}$	$M(U,0)=\emptyset$	$M(Q,0)=\{Q,V\}$	$M(Z,0)=\{Z\}$
$M(S,1)=\{Q,U\}$	$M(V,1)=\emptyset$	$M(U,1)=\{Z\}$	$M(Q,1)=\{Q,U\}$	$M(Z,1)=\{Z\}$

Диаграмма состояний распознавателя имеет следующий вид (рис.14):

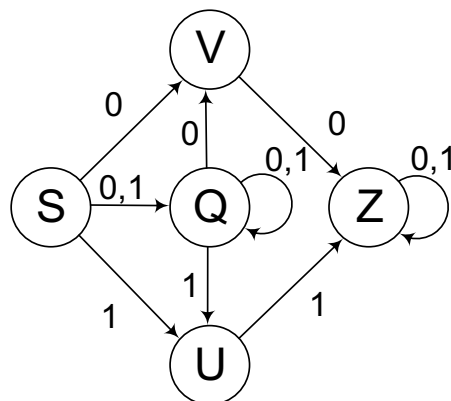


Рис.14. Диаграмма состояний примера НКР

Рассмотрим пошагово разбор входной цепочки данным недетерминированным конечным распознавателем:

Номер шага	Текущее состояние	Остаток входной цепочки	Возможные преемники	Выбор преемника
1	S	01001	V, Q	Q
2	Q	1001	U, Q	Q
3	Q	001	V, Q	V
4	V	01	Z	Z
5	Z	1	Z	Z

Заметим, что поиск правильного пути всегда связан с пошаговым выбором очередного состояния из нескольких альтернатив. Этот выбор не всегда такой очевидный, как в случае примера, а программная реализация недетерминированного конечного распознавателя является не такой уж тривиальной.

2.7. ПРЕОБРАЗОВАНИЕ НЕДЕТЕРМИНИРОВАННОГО КОНЕЧНОГО РАСПОЗНАВАТЕЛЯ В ДЕТЕРМИНИРОВАННЫЙ

Это преобразование основано на представлении альтернативных путей разбора недетерминированного распознавателя одним путем в эквивалентном детерминированном распознавателе (рис.15).

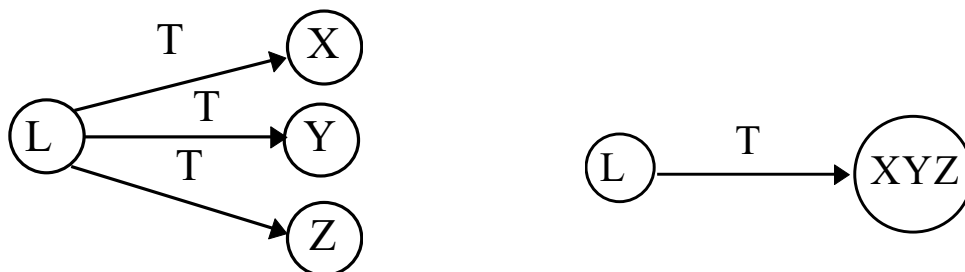


Рис.15. Схема замены альтернатив при преобразовании НКР в ДКР

Три альтернативных состояния X, Y, Z в НКР представляются одним состоянием $[XYZ]$ в ДКА, имя которого составлено из имен состояний НКР. Итак, если НКА представляется пятеркой

$$(K, V_T, M, S, Z),$$

то эквивалентный ДКА имеет вид

$$(K', V_T, M', S', Z').$$

Алфавит состояний K' определим через подмножества алфавита K . Элементы K' будем обозначать через $[S_1S_2...S_l]$, где $S_1, S_2, ..., S_l$ – имена состояний подмножества $\{S_1, S_2, ..., S_l\} \subset K$. Таким образом, число элементов в алфавите K' равно числу подмножеств алфавита K .

Функция переходов M образуется следующим образом. Если в НКР имеется частичная функция переходов вида $M(\{S_1, S_2, S_l, T\} = \{R_1, R_2, ..., R_t\})$, то частичная функция переходов эквивалентного ДКР будет $M([S_1S_2S_l], T) = [R_1R_2..., R_t]$.

Начальное состояние. Если в НКР множество начальных состояний $\{S_1, S_2, ..., S_p\}$, то начальным состоянием ДКР необходимо сделать состояние с именем $[S_1S_2...S_p]$.

Множество заключительных состояний Z' ДКА образуется из имен состояний, входящих в подмножество состояний НКР, в котором ИМЕЕТСЯ хотя бы одно заключительное состояние НКР из множества Z .

Как НКА, так и эквивалентный ему ДКА, построенный по ранее приведенным правилам, допускают одни и те же входные цепочки.

Тривиальный алгоритм построения ДКА заключается в следующем:

- 1) построить множество состояний K' путем перечисления всех подмножеств K ;
- 2) выделить из множества K' начальное состояние S' и множество заключительных состояний Z' ;
- 3) создать пустую таблицу переходов, строки которой помечать элементами из K' , а столбцы – символами входного алфавита;
- 4) заполнить таблицу переходов.

Тривиальный алгоритм хотя и прост, но трудоемок, а построенный ДКР практически всегда содержит большое число недостижимых состояний. Поэтому лучше при построении ДКА воспользоваться модифицированным алгоритмом, устраняющим сразу и недостижимые состояния.

Модифицированный алгоритм:

- 1) построить начальное состояние S' и отметить им первую строку таблицы переходов;

2) вычислить все состояния K' , которые могут быть достигнуты из данного состояния;

3) если во вновь вычисленных состояниях не встречаются такие, которые отмечали ранее строки таблицы переходов, то для них завести новые строки и повторить п.2;

4) выделить в таблице переходов заключительные состояния Z' .

Проведем построение ДКР для НКР, для чего перерисуем еще раз диаграмму состояний НКР (разд.3.3) (рис.16).

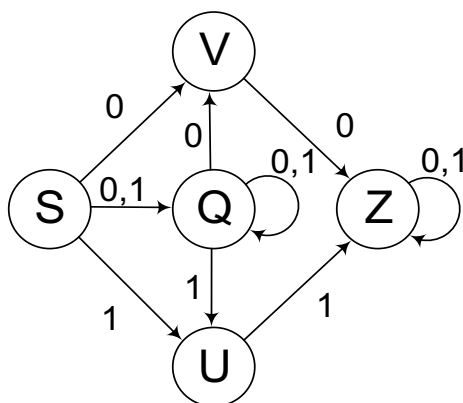


Рис. 16. Пример недетерминированного конечного распознавателя

Проведем пошаговое построение детерминированного конечного распознавателя.

1. $S' = [S]$.

	0	1	Y/N
[S]			

2. Заполним строку для S и допишем пустые строки для вновь образованных новых состояний.

	0	1	Y/N
[S]	[VQ]	[UQ]	
[VQ]			
[UQ]			

3. Продолжим вычисление состояний для строк $[VQ]$ и $[UQ]$ и допишем новые строки.

4. Продолжим вычисления состояний для строк $[VQZ]$, $[UQZ]$.

	0	1	Y/N
[S]	[VQ]	[UQ]	
[VQ]	[VQZ]	[UQ]	
[UQ]	[VQ]	[UQZ]	
[VQZ]			
[UQZ]			

	0	1	Y/N
[S]	[VB]	[UQ]	
[VQ]	[VQZ]	[UQ]	
[UQ]	[VQ]	[UQZ]	
[VQZ]	[VQZ]	[UQZ]	
[UQZ]	[VQZ]	[UQZ]	S

5. Новые состояния не образуются, поэтому дописывание новых строк прекращаем. Таблица переходов построена.

Теперь ДКР представляется следующей диаграммой состояний (рис.17.):

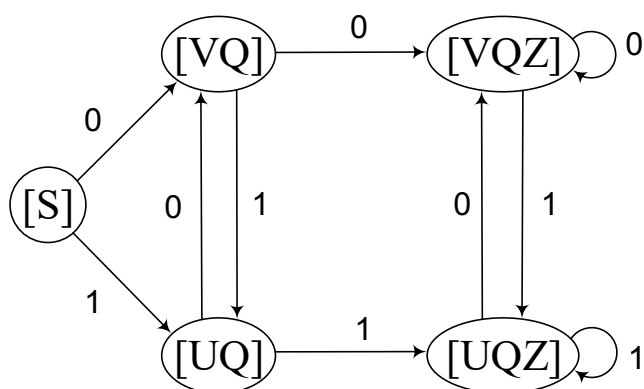


Рис. 17. Диаграмма состояний ДКР

Здесь $[S]$ – начальное состояние; $Z = \{[VQZ], [UQZ]\}$ – множество заключительных состояний.

Теперь минимизируем этот же детерминированный конечный распознаватель методом разбиения.

1. Построим начальное разбиение:

$$P_0 = (\underbrace{\{[S],[VQ],[UQ]\}}_{\text{1-й блок}}, \underbrace{\{[VQZ],[UQZ]\}}_{\text{2-й блок}})$$

2. Разобьем P_0 относительно входного символа 0, получим разбиение P_1 :

$$P_1 = (\{[S],[UQ]\}, \{[VQ]\}, \{[VQZ],[UQZ]\}).$$

3. Разобьем P_1 относительно 1, получим разбиение P_2 :

$$P_2 = (\{[S]\}, \{[UQ]\}, \{[VQ]\}, \{[VQZ],[UQZ]\}).$$

4. Разобьем P_2 относительно 0, получим разбиение P_3 :

$$P_3 = (\{[S]\}, \{[UQ]\}, \{[VQ]\}, \{[VQZ],[UQZ]\}).$$

Поскольку разбиение $P_1=P_2=P_3$, то дальнейшие действия по разбиению на блоки нужно прекратить. Состояния $[VQZ]$ и $[UQZ]$ являются эквивалентными.

Введем новые имена. Пусть $[S]$ соответствует S , $[VQZ]$ и $[UQZ]$ – K , $[VQ]$ – A , $[UQ]$ – B . После чего модифицированная таблица переходов минимизированного детерминированного конечного распознавателя примет вид

	0	1	Y/N
S	A	B	N
A	K	B	N
B	A	K	N
K	K	K	Y

Здесь H – начальное; K – заключительное состояния соответственно.

Теперь диаграмма состояний минимального детерминированного конечного распознавателя примет вид (рис.18)

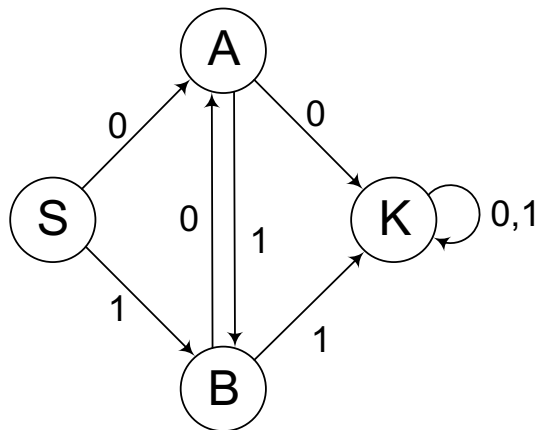


Рис. 18. Минимизированный конечный распознаватель

Как это видно из диаграммы состояний (рис.18), минимизированный детерминированный конечный распознаватель более прост в сравнении с исходным недетерминированным конечным распознавателем. В то же время разбираемые ими языки являются одинаковыми.

3. РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ И КОНЕЧНЫЕ РАСПОЗНАВАТЕЛИ

3.1. МОДИФИЦИРОВАННАЯ ФОРМА БЭКУСА – НАУРА И РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ

В гл. 1 и 2 были рассмотрены основные понятия и положения теории формальных грамматик, представленных в нотации Бэкуса – Наура и обсуждена их связь с конечными распознавателями. Рассмотрим еще одну нотацию грамматик, в которой дополнительно используются метасимволы $\{ \}$ – итерационные (фигурные) и $()$ – круглые скобки. Такая нотация грамматик называется модифицированной или расширенной нормальной бэкусовой формой (НБФ).

Запись $\{a\}$, где a – некоторая цепочка, означает, что речь идет о множестве цепочек, построенных из символа a , у которых a может отсутствовать (цепочка λ), либо повторяться любое число раз. Заметим, что в рассмотренных ранее грамматиках в нотации НБФ для порождения таких множеств цепочек служит рекурсия. Например, в грамматике $G[<\text{число}>]$ используется следующее леворекурсивное правило:

$$<\text{чис}> ::= <\text{чис}> <\text{цифра}> | <\text{цифра}> .$$

В модифицированной НБФ эквивалент данного правила по порождаемым цепочкам можно записать следующим образом:

$$<\text{чис}> ::= \{ <\text{цифра}> \} <\text{цифра}> .$$

Иногда для итерационных цепочек можно встретить запись вида $\{a\}_2^5$, у которой индексы определяют число повторяющихся цепочек, в нашем случае это множество цепочек, у которых a повторяется от двух до пяти раз.

Метасимволы «круглые скобки» несут такую же смысловую нагрузку, как и в обычных арифметических выражениях. Так, приведенные далее записи являются эквивалентными:

$$ab|ac|ad \longrightarrow a(b|c|d).$$

Теперь введем и рассмотрим понятия *регулярных выражений* (РВ) и *регулярных множеств* (РМ). Определим регулярные выражения конструктивно в виде алгебры регулярных выражений.

Пусть задан некоторый алфавит $S = \{s_1, s_2, \dots, s_n\}$, тогда:

- 1) $\emptyset, \lambda, s_1, s_2, \dots, s_n$ есть регулярные выражения (здесь $\emptyset$ пустое множество, λ – пустая цепочка);
- 2) если e – регулярное выражение, то (e) и $\{e\}$ также являются регулярными выражениями;
- 3) если e_1 и e_2 – регулярные выражения, то $e_1 e_2$ и $e_1 | e_2$ также регулярные выражения;
- 4) других регулярных выражений нет.

Регулярное выражение есть запись (формула), которая описывает (представляет) некоторое множество цепочек. Данное множество цепочек является значением регулярного выражения и называется регулярным множеством. Так, значением регулярного выражения e является регулярное множество. Значение регулярного выражения e обозначается как $|e|$.

Рассмотрим значение основных регулярных выражений:

$$\begin{aligned}
 |\emptyset| &= \emptyset; \\
 |\lambda| &= \lambda; \\
 |s_i| &= \{s_i\} \text{ – одноэлементное множество}; \\
 |(e)| &= |e|; \\
 |e_1 e_2| &= |e_1| \times |e_2| \\
 |e_1 | e_2| &= |e_1| \cup |e_2| \\
 |\{e\}| &= |e|^* \text{ – итерация множества } |e|.
 \end{aligned}$$

Сопоставляя модифицированную НБФ и регулярные выражения, можно заметить, что регулярные выражения и правые части правил грамматик модифицированной НБФ имеют одинаковое смысловое значение. Так, правило $E ::= a\{1\}$ и РВ $a\{1\}$ задают одно и то же множество цепочек вида $a, a1, a11, a111, \dots, a111\dots1\dots$

Однако построение конечного распознавателя из модифицированной НБФ по рассмотренному ранее в гл. 3 способу требует ее преобразования к НБФ, что не всегда тривиальная задача. Проиллюстрируем данное обстоятельство на конкретном примере.

Пусть имеется грамматика $G[Z]$ вида

$$Z ::= \{a|b\}c$$

и ее нужно привести к левосторонней НБФ, в которой правила являются автоматными, т.е. имеют вид $U ::= VT|T$, где T – терминал, а U и V – нетерминалы.

Введем нетерминал A и соответствующее ему правило вида

$$A ::= \{a|b\},$$

которое подставим в грамматику $G[Z]$, после чего получим новые правила

$$\begin{aligned} Z &::= Ac; \\ A &::= \{a|b\}. \end{aligned}$$

В новом представлении первое правило соответствует автоматному виду, а второе – не соответствует.

Продолжим преобразования со вторым правилом. Поскольку второе правило можно свести к бесконечному числу правил вида

$$\begin{aligned} A &::= \lambda & A &::= a & A &::= b & A &::= aa & A &::= ab \\ A &::= ba & A &::= bb & \text{и т.д.} \end{aligned}$$

Воспользовавшись рекурсией, сведем данные правила к пяти правилам вида

$$A ::= \lambda \quad A ::= a \quad A ::= b \quad A ::= Aa \quad A ::= Ab$$

Однако правила типа $A ::= \lambda$ в левосторонних автоматных грамматиках, представленных в нотации НБФ, не используются. Это правило задает нулевое вхождение A в правиле $Z ::= Ac$ грамматики $G[Z]$. Удалим правило $A ::= \lambda$, а чтобы не нарушать язык грамматики, введем новое правило $Z ::= c$ для компенсации удаляемого. После этого грамматика $G[Z]$ примет вид левосторонней автоматной грамматики

$$\begin{aligned} Z &::= Ac|c \\ A &::= a|b|Aa|Ab. \end{aligned}$$

Построенная по ранее сформулированному способу по данной грамматике диаграмма состояний представлена на рис.19

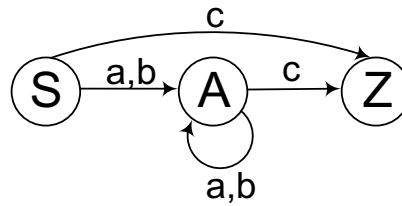


Рис.19. Диаграмма состояний конечного распознавателя

Предлагаем самостоятельно убедиться, что преобразование грамматики $Z ::= \{ \{a\}b \} c$ к эквивалентной левосторонней автоматной грамматике в нотации НБФ ранее рассмотренным способом не представляется возможным.

3.2. СИСТЕМА ПЕРЕХОДОВ

Рассмотрим модель конечного распознавателя, называемую системой переходов (СП). Система переходов имеет одно начальное S состояние и одно заключительное Z , а ее дуги взвешены регулярными выражениями.

Например, пусть e – некоторое регулярное выражение, тогда ему соответствует следующая система переходов (рис.20):

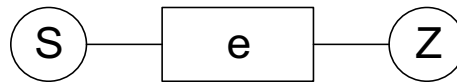
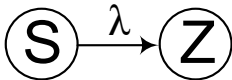
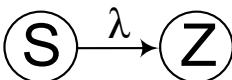
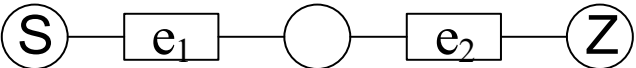
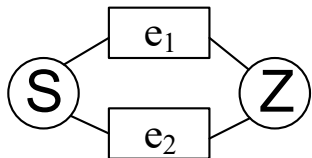
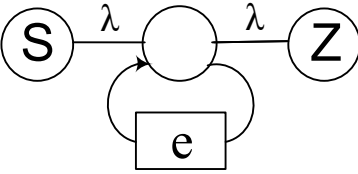


Рис.20. Система переходов

Работу такого распознавателя нужно понимать следующим образом: любая из цепочек, принадлежащая значению регулярного выражения e , переводит распознаватель из начального состояния S в конечное состояние Z и тем самым распознается.

Рассмотрим типовые базовые конструкции систем переходов для основных регулярных выражений, которые играют роль элементарных конструктивов. Все они приведены далее в таблице.

Регулярное выражение	Система переходов
$\emptyset$	
λ	
s_i	
$e_1 e_2$	
$e_1 e_2$	
$\{e\}$	

Поскольку составное регулярное выражение можно всегда представить в виде композиции более простых, то, проводя аналогичную процедуру над исходной системой переходов, можно всегда ее представить в виде композиции элементарных систем переходов.

Например, приведем по шагам построение системы переходов для регулярного выражения, как показано далее (рис.21).

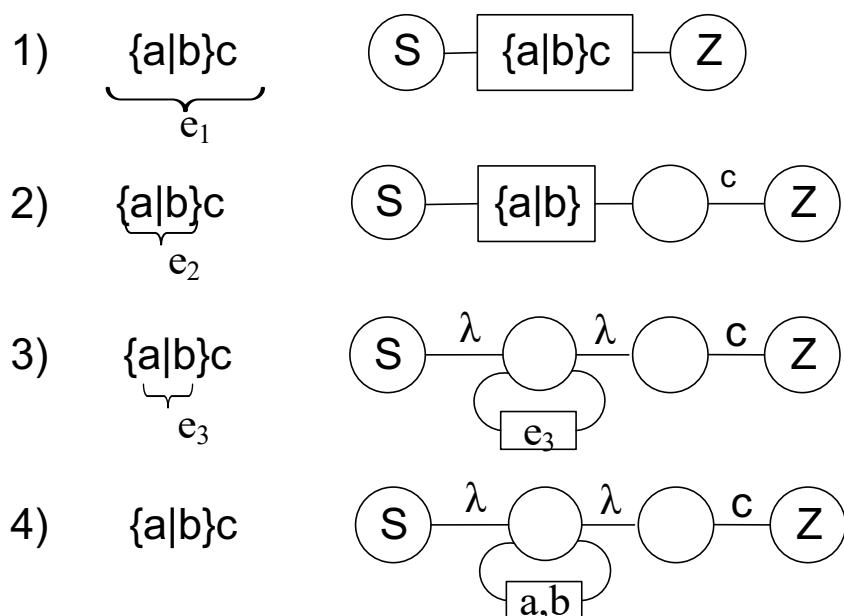


Рис. 1. Пример. Пошаговое построение системы переходов

Дальнейшая декомпозиция системы переходов невозможна.

Систему переходов, у которой дуги взвешены только символами алфавита и пустой цепочкой λ , будем называть приведенной и обозначать СП_п.

Смысл дуги λ . Если в системе переходов два состояния (A и B) связаны дугой λ , то в один и тот же момент времени может быть текущим как состояние A , так и состояние B .

Нетрудно убедиться, что распознаватель, представленный системой переходов, допускает все цепочки, порождаемые соответствующим регулярным выражением.

3.3. ПРЕОБРАЗОВАНИЕ СИСТЕМЫ ПЕРЕХОДОВ В НЕДЕТЕРМИНИРОВАННЫЙ КОНЕЧНЫЙ РАСПОЗНАВАТЕЛЬ

Поскольку между недетерминированным конечным распознавателем и системой переходов имеется глубинная связь, задаваемая разбираемым ими языком, то рассмотрим их взаимные эквивалентные преобразования.

Преобразование диаграммы состояний в систему переходов с учетом физического смысла дуги λ является тривиальным.

Пусть имеется диаграмма состояний недетерминированного конечного распознавателя, в которой $S_1, S_2, \dots, S_n$ – начальные состояния, а $Z_1, Z_2, \dots, Z_n$ – заключительные. Введем для эквивалентной системы переходов начальное состояние S и заключительное Z . Если из S провести λ -дуги во все $S_1, S_2, \dots, S_n$, а из $Z_1, Z_2, \dots, Z_n$ – в Z , то диаграмма состояний преобразуется в эквивалентную систему переходов (рис. 22):

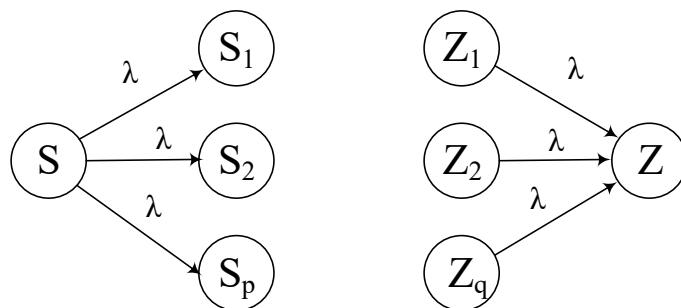


Рис. 22. Схема преобразования недетерминированного конечного распознавателя в систему переходов

Преобразование системы переходов в диаграмму состояний недетерминированного конечного распознавателя основывается на свойстве λ -дуг и связано с их последовательным исключением. Изложим алгоритм преобразования.

1. Найти в системе переходов некоторое состояние, из которого исходит λ -дуга. Продвигаться по λ -дугам из этого состояния до тех пор, пока не встретится ситуация, когда из состояния A в состояние B ведет λ -дуга, а из состояния B λ -дуг не исходит (рис. 23)



Рис. 23. Иллюстрация к п.1 алгоритма

Если такой ситуации не окажется, то в системе переходов содержится кольцевой λ -путь и следует перейти к п.2 алгоритма. Если такая ситуация есть, то нужно выполнить следующие действия:

1.1. Пусть в ситуации п.1 имеется в системе переходов следующий λ -путь (рис. 24).

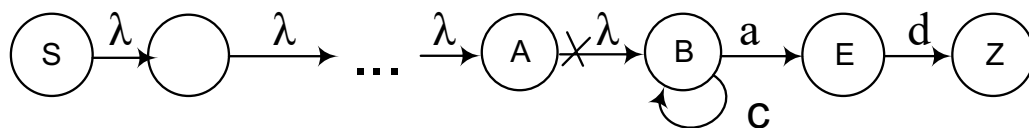


Рис. 24. Иллюстрация к п.1 алгоритма

Он начинается из начального состояния S и ведет в состояние B , тогда его последняя λ -дуга, ведущая в состояние B , удаляется, а на состояние A навешиваются все дуги, исходящие из состояния B . Состоянию B приписывается признак начального состояния. В результате получим следующий фрагмент системы переходов (рис.25):

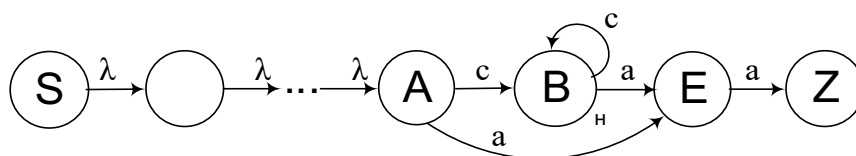


Рис. 25. Иллюстрация к п.1.1 алгоритма

1.2. Пусть в ситуации п.1 имеется путь вида (рис. 26)

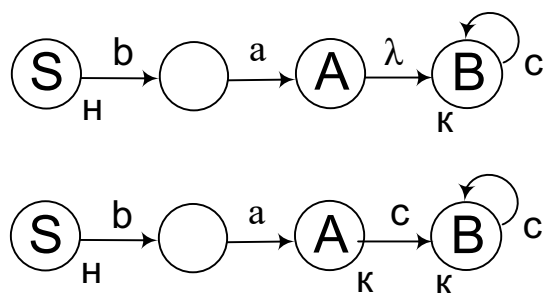


Рис. 26. Иллюстрация к п.1.2 алгоритма

Здесь λ -дуга ведет в конечное состояние B . Эта λ -дуга удаляется, а на состояние A навешиваются все дуги, исходящие из состояния B . Состоянию A приписывается признак конечного состояния.

1.3. Пусть A и B не конечные и не начальные состояния, тогда при удалении λ -дуги осуществляется только навешивание на состояние A всех дуг, исходящих из состояния B (рис.27).



Рис. 27. Иллюстрация к п.1.3 алгоритма

2. Если в системе переходов распознавателя отсутствует ситуация, отмеченная в п.1, то в ней обязательно имеется кольцевой λ -путь вида (рис. 28)

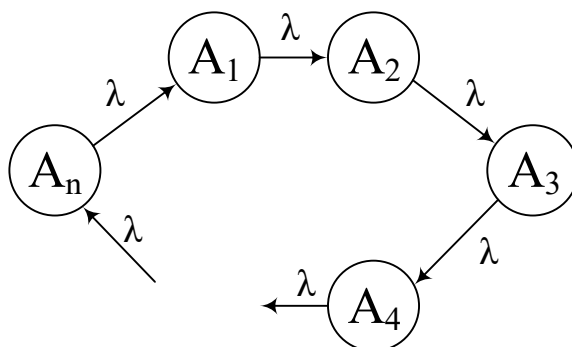


Рис. 28. Иллюстрация к п.2 алгоритма

В этом случае все состояния стягиваются в одно состояния A , а если среди состояний кольцевого λ -пути было хотя бы одно заключительное/начальное состояние, то A становится также заключительным/начальным состоянием.

Далее представлен пример, иллюстрирующий пункт 2 алгоритма (рис. 29).

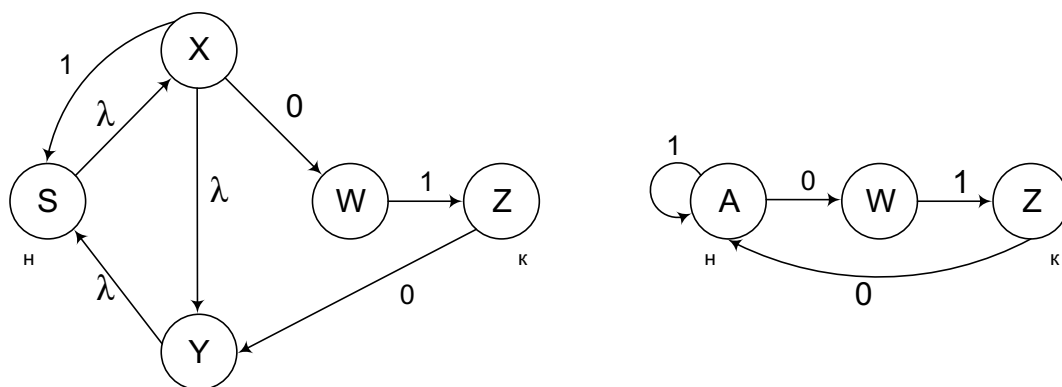


Рис.29. Пример преобразования кольцевого λ -пути

Заметим, что при переходе от системы переходов распознавателя к недетерминированному конечному распознавателю λ -дуги исключаются последовательно до тех пор, пока не останется ни одной λ -дуги.

Рассмотрим примеры преобразования систем переходов в недетерминированный распознаватель и далее его сведение к минимизированному детерминированному конечному распознавателю. Отметим, что все модели представления распознавателя эквивалентные, поскольку разбирают один и тот же язык.

Пример 1. Пусть задано регулярное выражение вида $\{a|b\}^*$. Покажем для него процесс построения минимального детерминированного конечного распознавателя.

Вначале построим приведенную систему переходов и преобразуем ее в недетерминированный конечный распознаватель.

На рис. 30 представлена соответствующая данному регулярному выражению приведенная система переходов и показан процесс удаления в ней λ -дуг. Результирующая диаграмма состояний не содержит λ -дуг и представляет недетерминированный конечный распознаватель.

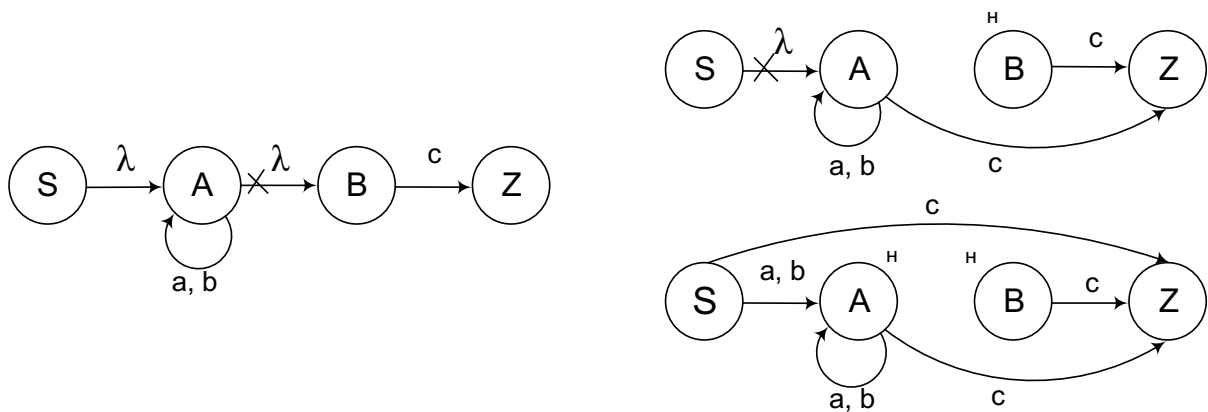


Рис. 30. Пример преобразования системы переходов в недетерминированный конечный распознаватель

Далее преобразуем недетерминированный конечный распознаватель в эквивалентный детерминированный конечный распознаватель.

Сперва сконструируем имя начального состояния $[SAB]$ и конечного состояния $[Z]$ соответственно предполагаемому детерминированному конечному распознавателю. Далее перейдем к заполнению таблицы переходов

	a	b	c
$[SAB]$	$[A]$	$[A]$	$[Z]$
$[A]$	$[A]$	$[A]$	$[Z]$
$[Z]$	---	---	---

Для компактности записи и простоты представления переименуем полученные состояния следующим образом:

$$[SAB] = S;$$

$$[A] = A;$$

$$[Z] = Z.$$

Далее построим новую таблицу переходов.

	a	b	c
S	A	A	Z
A	A	A	Z
Z	---	---	---

Данной таблице переходов соответствует следующая диаграмма состояний детерминированного конечного (рис. 31).

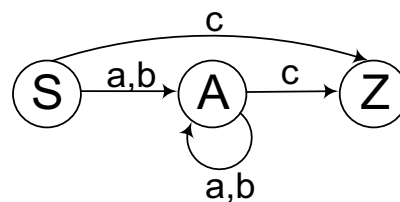


Рис.31. Диаграмма состояний детерминированного конечного распознавателя

В представленном детерминированном конечном распознавателе состояния S и A являются эквивалентными, поэтому они стягиваются в одно эквивалентное состояние S, после чего диаграмма состояний минимизированного конечного распознавателя примет следующий вид (рис.32):

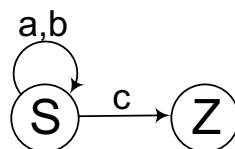


Рис.32. Диаграмма состояний минимизированного детерминированного конечного распознавателя

Пример 2. Пусть задано следующее регулярное выражение

$$\{a\{b\}\}c.$$

Процесс последовательного удаления λ -дуг в системе переходов и приведенная система переходов показаны на рис. 33.

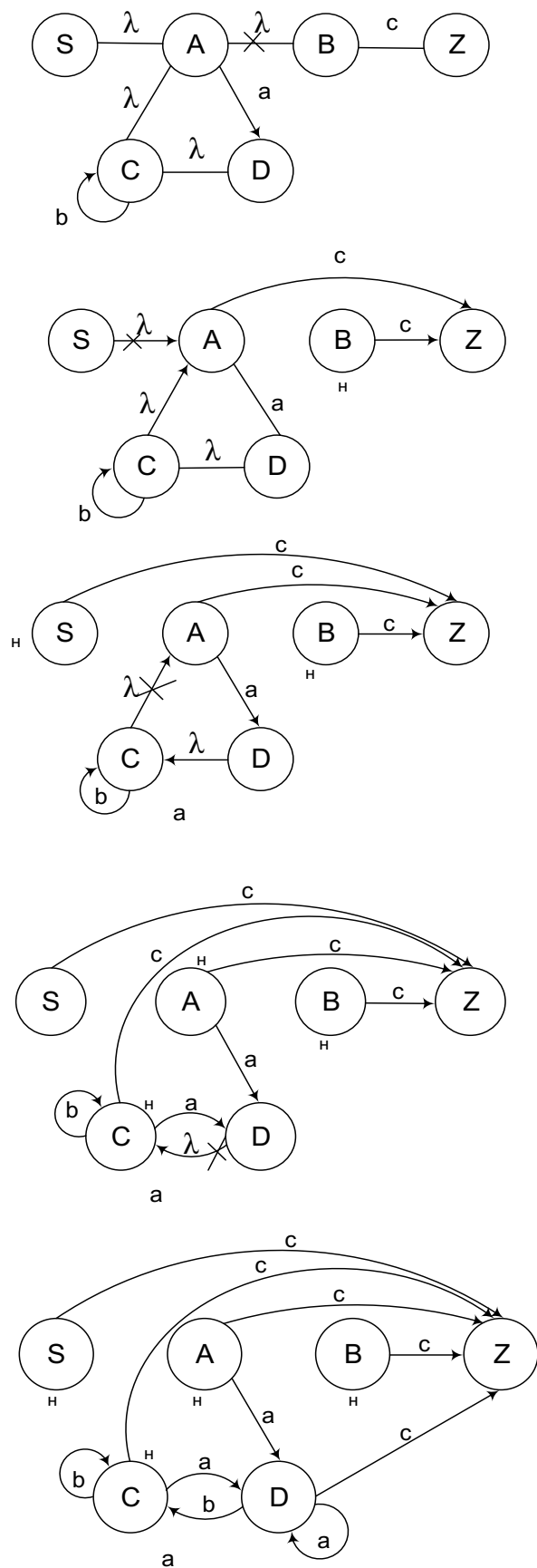


Рис.33. Удаления λ -дуг в системе переходов

Полученная на последнем шаге диаграмма состояний представляет недетерминированный конечный распознаватель.

Теперь для данного недетерминированного конечного распознавателя, используя рассмотренный ранее метод (2.7), построим эквивалентный детерминированный конечный распознаватель. Опуская рассуждения, приведем конечный результат, который представлен в нотации следующей таблицы переходов:

	a	b	c	Y/N
[SABCD]	[D]	[C]	[Z]	N
[D]	[D]	[C]	[Z]	N
[C]	[D]	[C]	[Z]	N
[Z]	---	---	---	Y

Переименуем в данной таблице состояние [SABCD] в S; [D]–B; [C]–C; [Z]–Z, после чего данная таблица переходов преобразуется к виду

	a	b	c	Y/N
S	D	C	Z	N
D	D	C	Z	N
C	D	C	Z	N
Z	---	---	---	Y

Соответственно диаграмма состояний эквивалентного детерминированного распознавателя, построенная по данной таблице переходов, приведена на рис. 34.

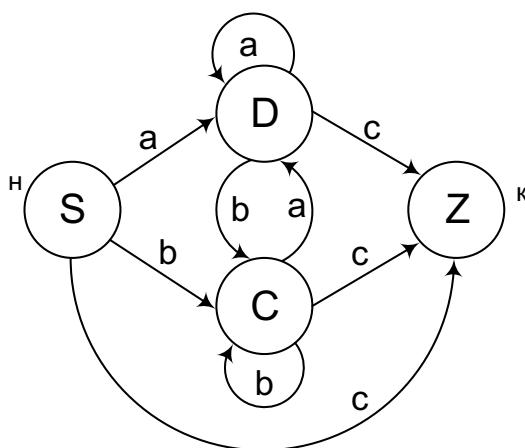


Рис.34. Детерминированный конечный распознаватель

Выполним процедуру минимизации детерминированного конечного распознавателя методом разбиения на блоки (2.5). Сперва построим начальное разбиение P_0 , которое имеет следующий вид:

$$P_0 = \{S, D, C\}, \{Z\}.$$

Расщепим блоки разбиения P_0 относительно входного символа a . Поскольку приемники состояний $\{S, D, C\}$ находятся внутри блока, то расщепление не происходит и разбиение P_0 не меняется, т.е. $P_1 = P_0$. Повторим аналогично расщепление разбиения P_0 относительно букв b и c . Видим, что блок $\{S, D, C\}$ не расщепляется относительно. Поэтому состояния блока $\{S, D, C\}$ эквивалентные и введем для блока новое эквивалентное имя S_e .

Тогда с учетом выполненных переименований минимизированный детерминированный конечный распознаватель примет следующий вид (рис. 35):

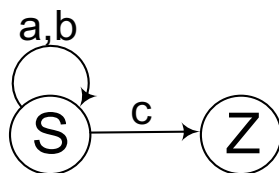


Рис.35. Минимальный детерминированный конечный распознаватель

Таким образом, нужно отметить, что формализованное построение распознавателя по грамматике, заданной в модифицированной НБФ, нужно проводить по схеме $PВ \rightarrow СП \rightarrow НКР \rightarrow ДКР \rightarrow \text{минимальный ДКР}$.

4. ПРЕОБРАЗОВАНИЕ РЕГУЛЯРНЫХ ВЫРАЖЕНИЙ НА ОСНОВЕ ОБРАТНОЙ ПОЛЬСКОЙ ЗАПИСИ

Одну из ключевых ролей в системном программировании играет обратная польская запись (ОПЗ), названная так в честь польского математика Я. Лукасевича, который впервые использовал ее в задачах математической логики. ОПЗ обладает двумя свойствами:

- 1) отсутствием скобок;
- 2) возможностью вычисления выражения путем однократного просмотра слева направо.

Благодаря данным свойствам ОПЗ широко применялась в первых компиляторах в качестве промежуточного языка для генерации машинного кода. Сами идеи такого промежуточного языка стали классикой и используются в том или ином виде в методиках трансляции.

4.1. ОСНОВНЫЕ ПОНЯТИЯ

Пусть задано арифметическое выражение

$$(a + b) * c + d / (e + b).$$

Представим его в виде дерева, в узлах которого разместим операции, а в концевых узлах – операнды (рис.36).

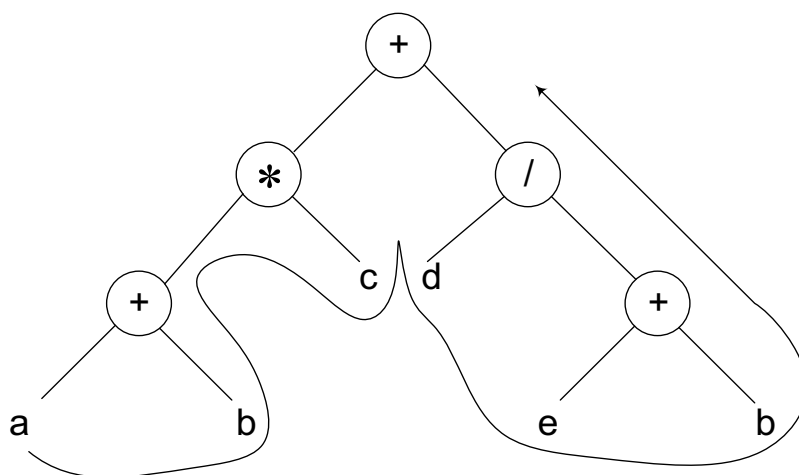


Рис.36. Дерево арифметического выражения

Если выполнить обход (просмотр) узлов данного дерева, начиная с самого левого концевой узла, и записывать последовательно в строку имена узлов, то последовательность символов будет представлять ОПЗ арифметического выражения. В обходе узел рассматривается после просмотра всех исходящих из него ветвей.

В приведенном примере арифметического выражения и его дерева (рис. 2) ОПЗ имеет вид

$$a \ b + c * d \ e \ b + / + .$$

Вычисление ОПЗ осуществляется по алгоритму:

1) просмотреть очередной символ ОПЗ (просмотр осуществляется слева направо);

2) если символ операнд, то просматривается следующий символ;

3) если символ – k-нарная операция, то из входной последовательности выбирается предшествующие k символов-операндов, над ними выполняется операция, результат размещается во временную переменную r_i и записывается на место самого первого операнда. Символ операции и все операнды, кроме модифицированного первого, из ОПЗ удаляются.

Далее в качестве примера представлено пошаговое вычисление приведенного выше ОПЗ арифметического выражения.

№ п/п	Состояние входной строки ОПЗ	Действие
1	<u>a</u> b + c * d e b + / +	Просмотреть следующий символ
2	a <u>b</u> + c * d e b + / +	Просмотреть следующий символ
3	a b <u>+</u> c * d e b + / +	$r_1 = a + b$
4	r_1 <u>c</u> * d e b + / +	Просмотреть следующий символ
5	r_1 c <u>*</u> d e b + / +	$r_1 = r_1 * c$
6	r_1 <u>d</u> e b + / +	Просмотреть следующий символ
7	r_1 d <u>e</u> b + / +	Просмотреть следующий символ
8	r_1 d e <u>b</u> + / +	Просмотреть следующий символ
9	r_1 d e b <u>+</u> / +	$r_2 = e + b$
10	r_1 d r_2 <u>/</u> +	$r_1 = d / r_2$
11	r_1 r_2 <u>+</u>	$r_1 = r_1 + r_2$
	Вычисление ОПЗ окончено	

Один из эффективных алгоритмов по переводу арифметических выражений в ОПЗ предложил голландский ученый Е.В. Дейкстра, алгоритм впоследствии получил название алгоритм Дейкстры. В его основе лежит стек и таблица приоритетов операций, которая выглядит следующим образом:

Операции, ограничители	Приоритет
(	0
)	1
+, -	2
*, /	3
!	4

Убывание приоритетов в таблице идет сверху вниз, здесь 0 – самый высокий приоритет, 4 – самый низкий. Данные приоритеты нельзя связывать с приоритетами операций традиционной арифметики, которые задают порядок действий при вычислении выражений.

На рис.37 схематично изображено действия алгоритма.

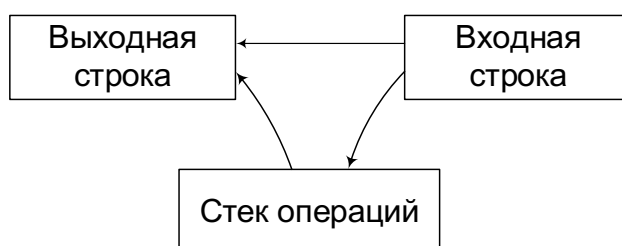


Рис.37. Схема работы алгоритма Дейкстры

Сам алгоритм Дейкстры включает три процедуры.

1. Входная строка просматривается посимвольно слева направо. Если символ – операнд, то он переписывается в выходную строку, если – операция, то выполняется следующая последовательность действий:

- при пустом стеке операция помещается в его вершину;
- если приоритет у анализируемого символа ниже, чем у операции в вершине стека, то он добавляется в стек;

– если приоритет у анализируемого символа равен или больше, чем у операции в вершине стека, то из стека в выходную строку последовательно выталкиваются все знаки с меньшим или равным приоритетом по сравнению с анализируемым символом. После этого входной просматриваемый символ добавляется в вершину стека.

2. Обработка скобок. Открывающая круглая скобка всегда записывается в вершину стека. Если анализируемый символ – закрывающая круглая скобка, то из стека выталкиваются все операции до открывающей скобки. Сами скобки уничтожаются и в выходную строку не попадают.

3. Если обработаны все символы входной строки, то оставшиеся знаки стека последовательно выталкиваются и дописываются в выходную строку.

Рассмотрим на примере развертку процесса получения ОПЗ.

Выходная строка		<i>a</i>		<i>b</i>	+		<i>c</i>	/	<i>d</i>			<i>e</i>		<i>b</i>	+	/+
Состояния стека	(	(	+	+		/	/	+	+	/	(	(	+	+	/	
Входная строка	(	<i>a</i>	+	<i>b</i>	)	/	<i>c</i>	+	<i>d</i>	/	(	<i>e</i>	+	<i>b</i>	)	

Во входной строке располагается исходное арифметическое выражение, в выходной – результат в виде ОПЗ. В средней строке отображается текущее состояние стека.

4.2. ОБРАТНАЯ ПОЛЬСКАЯ ЗАПИСЬ РЕГУЛЯРНОГО ВЫРАЖЕНИЯ

Для проведения построения конечных распознавателей большой размерности распространим понятие обратной польской записи на регулярные выражения. Для этого введем следующие операции в ОПЗ регулярного выражения:

Операция	Комментарий
*	Операция сцепления. Если $e_1 * e_2$, то $ e_1 * e_2 = e_1 \times e_2 $
	Операция ИЛИ. Если $e_1 e_2$, то $ e_1 e_2 = e_1 \cup e_2 $
}	Одноместная (унарная) операция итерации, которая эквивалентна { }. Так, РВ {a} будет иметь запись вида a}

Проиллюстрируем построение ОПЗ на примере регулярного выражения $e = \{a/b\}c$. Вначале построим дерево и выполним его обход (рис.38):

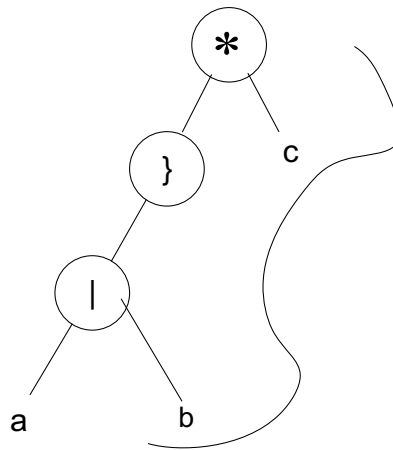


Рис.38. Дерево регулярного выражения $\{a/b\}c$ и его обход

Теперь сконструируем преобразование регулярного выражения в обратную польскую запись на основе алгоритма Дейкстры, для чего модифицируем алгоритм следующим образом. Вначале определим приоритеты операции и сформируем таблицу приоритетов:

Операции, ограничители	Приоритет
(, {	0
), }	1
	2
*	3

Определим следующие правила обработки операций:

1. Круглые скобки обрабатываются так же, как и в традиционном алгоритме Дейкстры.
2. Открывающая фигурная скобка { размещается в вершину стека.
3. Закрывающая фигурная скобка } выталкивает из стека все знаки операций до ближайшей { включительно, а также выталкивает в выходную строку саму операцию }.

Теперь проиллюстрируем алгоритм на примере следующего регулярного выражения:

$$e = \{a|b\}cd(b|d) .$$

Вначале введем в выражение явно операцию сцепления *, после чего оно примет вид

$$e = \{a|b\} * c * d * (b|d) .$$

Развертка процесса получения ОПЗ данного выражения представлена таблицей:

Выходная строка		<i>a</i>		<i>b</i>	}		<i>c</i>	*	<i>d</i>	*		<i>b</i>		<i>d</i>		*
Состояния стека	{	{				*	*	*	*	*	(	(			*	
Входная строка	{	<i>a</i>		<i>b</i>	}	*	<i>c</i>	*	<i>d</i>	*	(	<i>b</i>		<i>d</i>	)	

Таким образом, ОПЗ регулярного выражения *e* имеет вид

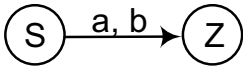
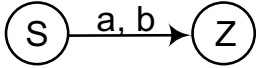
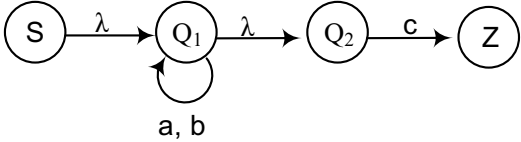
$$ab| \} c * d * b d | *$$

и совпадает с предыдущим результатом.

4.3. ИНТЕРПРЕТАЦИЯ ВЫЧИСЛЕНИЯ ОПЗ РЕГУЛЯРНОГО ВЫРАЖЕНИЯ.

Следует сразу оговориться, что вычисление ОПЗ регулярного выражения будем интерпретировать как построение системы переходов. Без потери общности общие положения проиллюстрируем на конкретном примере.

Пусть задано ОПЗ предыдущего примера, и для него запишем пошаговую процедуру вычисления ОПЗ.

№ п/п	Состояние входной строки ОПЗ	Действие	Результат
1	$\underline{a}b \}c*d*bd *$	Просмотреть следующий символ	
2	$a\underline{b} \}c*d*bd *$	Просмотреть следующий символ	
3	$ab \underline{_}\}c*d*bd *$	Построить систему переходов $R_1 = a b$	
4	$R_1 \underline{_}\}c*d*bd *$	Выполнить итерацию над R_1 $R_1 = \{R_1\}$	
5	$R_1c*\underline{_}\}d*bd *$	Просмотреть следующий символ	
6	$R_1c*d*\underline{_}\}bd *$	Построить систему переходов $R_1 = R_1 * c$	

7	$R_1 \underline{d} * b d ^*$	Просмотреть следующий символ	
8	$R_1 \underline{d} * b d ^*$	Построить систему переходов $R_1 = R_1 * b$	<pre> graph LR S((S)) -- λ --> Q1((Q1)) Q1 -- λ --> Q2((Q2)) Q2 -- c --> Q3((Q3)) Q3 -- d --> Z((Z)) Q1 -- "a, b" --> Q1 </pre>
9	$R_1 \underline{b} d ^*$	Просмотреть следующий символ	
10	$R_1 \underline{b} d ^*$	Просмотреть следующий символ	
11	$R_1 \underline{b} d ^*$	Построить систему переходов $R_2 = b d$	<pre> graph LR S((S)) -- "b d" --> Z((Z)) </pre>
12	$R_1 R_2^*$	Построить систему переходов $R_1 = R_1 * R_2$	<pre> graph LR S((S)) -- λ --> Q1((Q1)) Q1 -- λ --> Q2((Q2)) Q2 -- c --> Q3((Q3)) Q3 -- d --> Q4((Q4)) Q4 -- "b d" --> Z((Z)) Q1 -- "a, b" --> Q1 </pre>
Вычисление ОПЗ окончено			

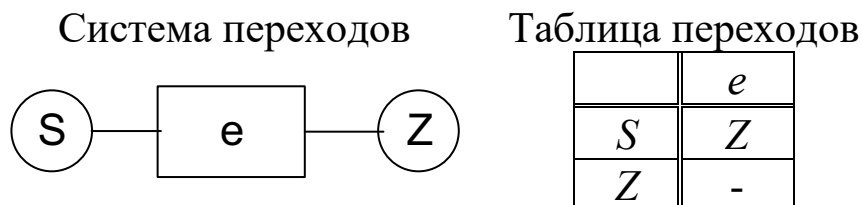
Заметим, что, во-первых, в процессе вычисления системы переходов образовано множество состояний $Q = \{Q_1, Q_2, Q_2, Q_2\}$, во-вторых, введено множество временных переменных $R = \{R_1, R_2\}$, в которые размещаются промежуточные результаты вычислений. Промежуточными результатами являются фрагменты системы переходов.

В рассмотренном случае иллюстрация вычисления операций выполнялась диаграммами состояний, в то же время для систем переходов большой размерности представление системы переходов в нотации диаграммы состояний теряет наглядность, а более подходит нотация представления в виде таблицы переходов.

Рассмотрим алгоритм Дейкстры для случая, когда система переходов представлена в виде таблицы переходов. В этом случае

значениями временных переменных R_i являются частичные таблицы переходов, поэтому следует определить операции над такими таблицами переходов.

Пусть задано некоторое регулярное выражение e , для которого система переходов и таблица переходов имеют вид



Рассмотрим операцию сцепления $*$:

РВ	Таблица пе- реходов			Система переходов													
e_1		$\downarrow e_1 \downarrow$															
	S	Z															
	Z																
e_2		$\downarrow e_1 \downarrow$															
	S	Z															
	Z																
$e_1 * e_2$	<table><tr><td></td><td>$\downarrow e_1 \downarrow$</td><td>$\downarrow e_2 \downarrow$</td></tr><tr><td>S</td><td>Q1</td><td>-</td></tr><tr><td>Q1</td><td>-</td><td>Z</td></tr><tr><td>Z</td><td>-</td><td>-</td></tr></table>				$\downarrow e_1 \downarrow$	$\downarrow e_2 \downarrow$	S	Q1	-	Q1	-	Z	Z	-	-		
					$\downarrow e_1 \downarrow$	$\downarrow e_2 \downarrow$											
				S	Q1	-											
				Q1	-	Z											
				Z	-	-											
Объединяем состояния																	
Результат объединения																	

Схематично работа с таблицами в случае операции сцепления представлена на рис.39:

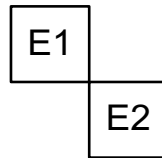


Рисунок 39. Схема соединения таблиц для операции *

Теперь рассмотрим операцию | (ИЛИ):

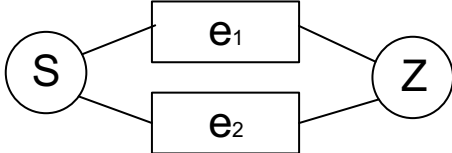
Регулярное выражение	Таблица переходов		Система переходов								
e_1	<table><tr><td></td><td>e_1</td></tr><tr><td>S</td><td>Z</td></tr><tr><td>Z</td><td></td></tr></table>		$ e_1 $	S	Z	Z					
	$ e_1 $										
S	Z										
Z											
e_2	<table><tr><td></td><td>e_2</td></tr><tr><td>S</td><td>Z</td></tr><tr><td>Z</td><td></td></tr></table>		$ e_2 $	S	Z	Z					
	$ e_2 $										
S	Z										
Z											
$e_1 e_2$	<table><tr><td></td><td>e_1</td><td>e_2</td></tr><tr><td>S</td><td>Z</td><td>Z</td></tr><tr><td>Z</td><td>-</td><td>-</td></tr></table>		$ e_1 $	$ e_2 $	S	Z	Z	Z	-	-	Объединяем начальные и конечные состояния $e_1 e_2$  Результат объединения
	$ e_1 $	$ e_2 $									
S	Z	Z									
Z	-	-									

Схема выполнения операции ИЛИ представлена на рис. 40.

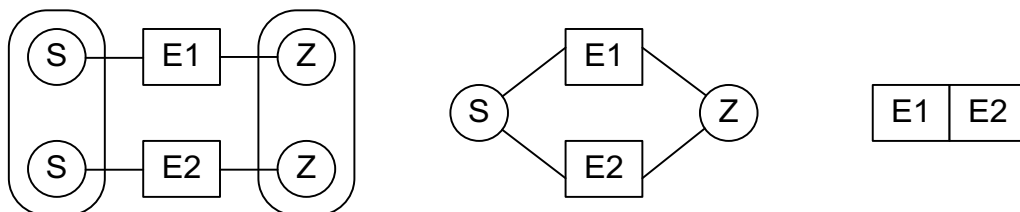
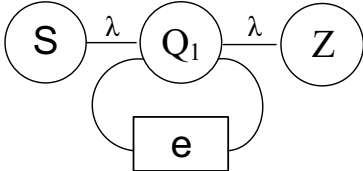


Рис. 40. Схема соединения таблиц для операции ИЛИ

Рассмотрим операцию итерации $\}$, которая представлена далее:

РВ	Таблица пе- реходов		Система переходов											
e	<table><tr><td></td><td><i>e</i></td></tr><tr><td><i>S</i></td><td><i>Z</i></td></tr><tr><td><i>Z</i></td><td>—</td></tr></table>		<i>e</i>	<i>S</i>	<i>Z</i>	<i>Z</i>	—							
	<i>e</i>													
<i>S</i>	<i>Z</i>													
<i>Z</i>	—													
e}	<table><tr><td></td><td><i>e</i></td><td>λ</td></tr><tr><td><i>S</i></td><td>—</td><td><i>Q</i>₁</td></tr><tr><td><i>Q</i>₁</td><td><i>Q</i>₁</td><td><i>Z</i></td></tr><tr><td><i>Z</i></td><td>—</td><td>—</td></tr></table>		<i>e</i>	λ	<i>S</i>	—	<i>Q</i> ₁	<i>Q</i> ₁	<i>Q</i> ₁	<i>Z</i>	<i>Z</i>	—	—	
	<i>e</i>	λ												
<i>S</i>	—	<i>Q</i> ₁												
<i>Q</i> ₁	<i>Q</i> ₁	<i>Z</i>												
<i>Z</i>	—	—												

Здесь во всех операциях Q_1 является очередным свободным именем из списка свободных имен состояний Q .

Теперь проиллюстрируем выполнение операций на конкретных примерах. Пусть заданы две системы переходов в виде таблиц R_1 и R_2 :

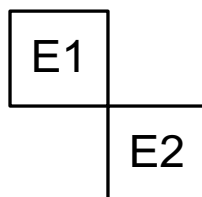
R_1	0	1
S	Q_1, Z	—
Q_1	—	Z
Z	—	—

R_2	0	1
S	—	Q_2, Q_3
Q_2	Z	—
Q_3	Z	—
Z	Z	Q_2, Q_3

Выполним операцию сцепления $R = R_1 * R_2$.

1. Как показано далее, построим новую итоговую таблицу переходов по схеме:

Схема операции сцепления *



Результат

R	0	1	0	1
S	Q_1, Z			
Q_1		Z		
Z				
S				Q_2, Q_3
Q_2			Z	
Q_3			Z	
Z			Z	Q_2, Q_3

2. Введем новое имя Q_4 для состояния Z в таблице R_1 и состояния S в таблице R_2 , получим:

R	0	1	0	1
S	Q_1, Q_4			
Q_1		Q_4		
Q_4				
Q_4				Q_2, Q_3
Q_2			Z	
Q_3			Z	
Z			Z	Q_2, Q_3

3. Объединим одинаковые строки Q_4 :

R	0	1	0	1
S	Q_1, Q_4			
Q_1		Q_4		
Q_4				Q_2, Q_3
Q_2			Z	
Q_3			Z	
Z			Z	Q_2, Q_3

4. Объединим одинаковые столбцы:

R	0	1
S	Q_1, Q_4	
Q_1		Q_4
Q_4		Q_2, Q_3
Q_2	Z	
Q_3	Z	
Z	Z	Q_2, Q_3

Теперь проиллюстрируем выполнение операции $|$ (ИЛИ) и построим таблицу переходов для выражения $R_1|R_2$.

1. Построим новую таблицу по схеме:

Схема операции | ИЛИ

R_1	R_2
-------	-------

Результат

R	0	1	0	1
S	Q_1, Z			Q_2, Q_3
Q_1		Z		
Q_2			Z	
Q_3			Z	
Z			Z	Q_2, Q_3

2. Объединяя одинаковые столбцы, получим:

	0	1
S	Q_1, Z	Q_2, Q_3
Q_1		Z
Q_2	Z	
Q_3	Z	
Z	Z	Q_2, Q_3

Рассмотрим операцию итерации $\{R_1\}$:

1. Выберем очередное свободное имя Q_2 из списка имен Q и переименуем состояния S и Z . Получим таблицу R_1 вида

R_1	0	1
Q_2	Q_1, Q_2	
Q_1		Q_2
Q_2		

2. Объединим одноименные строки:

R_1	0	1
Q_2	Q_1, Q_2	
Q_1		Q_2

3. Построим новую таблицу R по схеме:

Схема операции }			Объединение				Результат			
R_1		λ	R_1	0	1	λ	R_1	0	1	λ
S		Q_2	S			Q_2	S			Q_2
	R_1		Q_2	Q_1, Q_2			Q_1		Q_2	
			Q_1		Q_2		Q_2	Q_1, Q_2		Z
Q_2		Z	Q_2			Z	Z			
Z			Z							

После объединения одноименных столбцов и одноименных строк получается искомый результат.

В качестве комплексного примера рассмотрим вычисление ОПЗ следующего регулярного выражения:

$$a b | \} c * d * b d | * .$$

С целью упрощения просмотр операндов не показан, приведена только обработка операторов. Представим пошаговый процесс построения системы переходов в табличной форме.

№ п/п	Остаток ОПЗ	Действия									
1	$ab \}c*d*bd *$	$R_1 = a b$ Построить таблицу с именем R_1 <table> <tr> <td>R_1</td><td>a</td><td>b</td></tr> <tr> <td>S</td><td>Z</td><td>Z</td></tr> <tr> <td>Z</td><td></td><td></td></tr> </table>	R_1	a	b	S	Z	Z	Z		
R_1	a	b									
S	Z	Z									
Z											
2	$R_1\}c*d*bd *$	$R_1 = \{R_1\}$ Выбрать новое состояние Q_1 из списка Q .									

		Переименовать S и Z в таблице R_1 в Q_1 <table><tr><td>R_1</td><td>a</td><td>b</td></tr><tr><td>Q_1</td><td>Q_1</td><td>Q_1</td></tr><tr><td>Q_1</td><td></td><td></td></tr></table>	R_1	a	b	Q_1	Q_1	Q_1	Q_1													
R_1	a	b																				
Q_1	Q_1	Q_1																				
Q_1																						
		Объединить одинаковые строки <table><tr><td>R_1</td><td>a</td><td>b</td></tr><tr><td>Q_1</td><td>Q_1</td><td>Q_1</td></tr></table>	R_1	a	b	Q_1	Q_1	Q_1														
R_1	a	b																				
Q_1	Q_1	Q_1																				
		Построить новую таблицу <table><tr><td></td><td>a</td><td>b</td><td>λ</td></tr><tr><td>S</td><td></td><td></td><td>Q_1</td></tr><tr><td>Q_1</td><td>Q_1</td><td>Q_1</td><td></td></tr><tr><td>Q_1</td><td></td><td></td><td>Z</td></tr><tr><td>Z</td><td></td><td></td><td></td></tr></table>		a	b	λ	S			Q_1	Q_1	Q_1	Q_1		Q_1			Z	Z			
	a	b	λ																			
S			Q_1																			
Q_1	Q_1	Q_1																				
Q_1			Z																			
Z																						
		Объединить одинаковы строки <table><tr><td></td><td>a</td><td>b</td><td>λ</td></tr><tr><td>S</td><td></td><td></td><td>Q_1</td></tr><tr><td>Q_1</td><td>Q_1</td><td>Q_1</td><td>Z</td></tr><tr><td>Z</td><td></td><td></td><td></td></tr></table>		a	b	λ	S			Q_1	Q_1	Q_1	Q_1	Z	Z							
	a	b	λ																			
S			Q_1																			
Q_1	Q_1	Q_1	Z																			
Z																						

3

$R_1 c * d * b d \mid *$

$R_1 = R_1 * c$
Дописать к матрице R_1 матрицу C

	a	b	λ	c
S			Q_1	
Q_1	Q_1	Q_1	Z	
Z				
S				Z
Z				

Выбрать новое состояние Q_2 из Q .
Переименовать в таблице R_1 состояние Z и в таблице C состояние S в состояние Q_2

	a	b	λ	c
S			Q_1	
Q_1	Q_1	Q_1	Q_2	
Q_2				
Q_2				Z
Z				

Объединить одинаковые строки

	a	b	λ	c
S			Q_1	
Q_1	Q_1	Q_1	Q_2	
Q_2				Z
Z				

4	$R_1 d * b d \mid *$	$R_1 = R_1 * d$ Дописать к таблице R_1 таблицу D <table><tr><td></td><td>a</td><td>b</td><td>λ</td><td>c</td><td>d</td></tr><tr><td>S</td><td></td><td></td><td>Q_1</td><td></td><td></td></tr><tr><td>Q_1</td><td>Q_1</td><td>Q_1</td><td>Q_2</td><td></td><td></td></tr><tr><td>Q_2</td><td></td><td></td><td></td><td>Z</td><td></td></tr><tr><td>Z</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>S</td><td></td><td></td><td></td><td></td><td>Z</td></tr><tr><td>Z</td><td></td><td></td><td></td><td></td><td></td></tr></table> Объединить состояния Z в R_1 и состояние S в D и переименовать их в Q_3. Объединить одноименные строки, после чего получим следующую таблицу: <table><tr><td></td><td>a</td><td>b</td><td>λ</td><td>c</td><td>d</td></tr><tr><td>S</td><td></td><td></td><td>Q_1</td><td></td><td></td></tr><tr><td>Q_1</td><td>Q_1</td><td>Q_1</td><td>Q_2</td><td></td><td></td></tr><tr><td>Q_2</td><td></td><td></td><td></td><td>Q_3</td><td></td></tr><tr><td>Q_3</td><td></td><td></td><td></td><td></td><td>Z</td></tr><tr><td>Z</td><td></td><td></td><td></td><td></td><td></td></tr></table>		a	b	λ	c	d	S			Q_1			Q_1	Q_1	Q_1	Q_2			Q_2				Z		Z						S					Z	Z							a	b	λ	c	d	S			Q_1			Q_1	Q_1	Q_1	Q_2			Q_2				Q_3		Q_3					Z	Z					
	a	b	λ	c	d																																																																											
S			Q_1																																																																													
Q_1	Q_1	Q_1	Q_2																																																																													
Q_2				Z																																																																												
Z																																																																																
S					Z																																																																											
Z																																																																																
	a	b	λ	c	d																																																																											
S			Q_1																																																																													
Q_1	Q_1	Q_1	Q_2																																																																													
Q_2				Q_3																																																																												
Q_3					Z																																																																											
Z																																																																																
5	$R_1 b d \mid *$	$R_2 = b \mid d$ Построить таблицу для R_2 <table><tr><td>R_2</td><td>b</td><td>d</td></tr><tr><td>S</td><td>Z</td><td>Z</td></tr><tr><td>Z</td><td></td><td></td></tr></table>	R_2	b	d	S	Z	Z	Z																																																																							
R_2	b	d																																																																														
S	Z	Z																																																																														
Z																																																																																

6	$R_I R_I *$	$R_1 = R_1 * R_2$ Дописать к таблице R_1 таблицу R_2 <table><tr><td></td><td>a</td><td>b</td><td>λ</td><td>c</td><td>d</td><td>b</td><td>d</td></tr><tr><td>S</td><td></td><td></td><td>Q_1</td><td></td><td></td><td></td><td></td></tr><tr><td>Q_1</td><td>Q_1</td><td>Q_1</td><td>Q_2</td><td></td><td></td><td></td><td></td></tr><tr><td>Q_2</td><td></td><td></td><td></td><td>Q_3</td><td></td><td></td><td></td></tr><tr><td>Q_3</td><td></td><td></td><td></td><td></td><td>Z</td><td></td><td></td></tr><tr><td>Z</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>S</td><td></td><td></td><td></td><td></td><td></td><td>Z</td><td>Z</td></tr><tr><td>Z</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table> Выбрать новое состояние Q_4 из Q. Переименовать в матрице R_1 состояние Z и в матрице R_2 состояние S в состояние Q_4. Объединить одинаковые строки <table><tr><td></td><td>a</td><td>b</td><td>λ</td><td>c</td><td>d</td><td>b</td><td>d</td></tr><tr><td>S</td><td></td><td></td><td>Q_1</td><td></td><td></td><td></td><td></td></tr><tr><td>Q_1</td><td>Q_1</td><td>Q_1</td><td>Q_2</td><td></td><td></td><td></td><td></td></tr><tr><td>Q_2</td><td></td><td></td><td></td><td>Q_3</td><td></td><td></td><td></td></tr><tr><td>Q_3</td><td></td><td></td><td></td><td></td><td>Q_4</td><td></td><td></td></tr><tr><td>Q_4</td><td></td><td></td><td></td><td></td><td></td><td>Z</td><td>Z</td></tr><tr><td>Z</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table> Объединить одинаковые столбцы <table><tr><td></td><td>a</td><td>b</td><td>c</td><td>d</td><td>λ</td><td></td><td>a</td></tr><tr><td>S</td><td></td><td></td><td></td><td></td><td>Q_1</td><td>S</td><td></td></tr><tr><td>Q_1</td><td>Q_1</td><td>Q_1</td><td></td><td></td><td>Q_2</td><td>Q_1</td><td>Q_1</td></tr><tr><td>Q_2</td><td></td><td></td><td>Q_3</td><td></td><td></td><td>Q_2</td><td></td></tr><tr><td>Q_3</td><td></td><td></td><td></td><td>Q_4</td><td></td><td>Q_3</td><td></td></tr><tr><td>Q_4</td><td></td><td>Z</td><td></td><td>Z</td><td></td><td>Q_4</td><td></td></tr><tr><td>Z</td><td></td><td></td><td></td><td></td><td></td><td>Z</td><td></td></tr></table>		a	b	λ	c	d	b	d	S			Q_1					Q_1	Q_1	Q_1	Q_2					Q_2				Q_3				Q_3					Z			Z								S						Z	Z	Z									a	b	λ	c	d	b	d	S			Q_1					Q_1	Q_1	Q_1	Q_2					Q_2				Q_3				Q_3					Q_4			Q_4						Z	Z	Z									a	b	c	d	λ		a	S					Q_1	S		Q_1	Q_1	Q_1			Q_2	Q_1	Q_1	Q_2			Q_3			Q_2		Q_3				Q_4		Q_3		Q_4		Z		Z		Q_4		Z						Z	
	a	b	λ	c	d	b	d																																																																																																																																																																											
S			Q_1																																																																																																																																																																															
Q_1	Q_1	Q_1	Q_2																																																																																																																																																																															
Q_2				Q_3																																																																																																																																																																														
Q_3					Z																																																																																																																																																																													
Z																																																																																																																																																																																		
S						Z	Z																																																																																																																																																																											
Z																																																																																																																																																																																		
	a	b	λ	c	d	b	d																																																																																																																																																																											
S			Q_1																																																																																																																																																																															
Q_1	Q_1	Q_1	Q_2																																																																																																																																																																															
Q_2				Q_3																																																																																																																																																																														
Q_3					Q_4																																																																																																																																																																													
Q_4						Z	Z																																																																																																																																																																											
Z																																																																																																																																																																																		
	a	b	c	d	λ		a																																																																																																																																																																											
S					Q_1	S																																																																																																																																																																												
Q_1	Q_1	Q_1			Q_2	Q_1	Q_1																																																																																																																																																																											
Q_2			Q_3			Q_2																																																																																																																																																																												
Q_3				Q_4		Q_3																																																																																																																																																																												
Q_4		Z		Z		Q_4																																																																																																																																																																												
Z						Z																																																																																																																																																																												
7	Вычисление окончено																																																																																																																																																																																	

Система переходов, построенная по таблице переходов, совпадает с предыдущим результатом.

Таким образом, расширение понятия обратной польской записи на регулярные выражения позволяет достаточно просто и эффективно организовать процесс построения систем переходов для регулярных выражений произвольной сложности.

Дальнейшее преобразование табличной системы переходов в недетерминированный конечный распознаватель и далее в детерминированный, а также минимизация проводится по рассмотренным ранее алгоритмам, правилам и процедурам.

Обучающемуся в качестве упражнения предлагается самостоятельно выполнить данные преобразования над рассмотренным ранее примером.

5. АВТОМАТЫ С МАГАЗИННОЙ ПАМЯТЬЮ

Модель конечного распознавателя, введенная ранее, может решать ограниченный круг задач, требующих фиксированного объема памяти. Однако в задачах, подобных обработке парных скобок в арифметических выражениях, такая модель не работает. В этом случае используют более мощную модель автомата с магазинной памятью (МП-автомат), обсуждению которой посвящен настоящий раздел.

5.1. НЕРЕГУЛЯРНЫЕ МНОЖЕСТВА ЦЕПОЧЕК

В гл.4 были описаны регулярные множества цепочек. Они являются результатом проведения операции $*$, $|$, $\{ \}$ над символами некоторого алфавита. Дополним эти операции еще двумя операциями, позволяющими получать нерегулярные множества.

Степень над символом алфавита. Степень указывает число повторений символа. Например, множество

$$\{1^n 0^n; n > 0\}$$

включает следующие цепочки: 10, 1100, ..., 111...1000...0, для $n=1, 2, \dots$ соответственно.

Для множества $\{(11)^n (00)^m; n > 0, m > 0\}$ цепочки имеют вид

$$\begin{array}{ll} 1100 & n=m=1 \\ 11111100 & n=3; m=1. \end{array}$$

Операция обращения цепочки. Если w – некоторая цепочка abc , то ее обращением является цепочка cba , которая обозначается w^r . Здесь r – знак обращения цепочки.

Например, цепочки множества

$$\{ ww^r \mid w = \{0|1\}^* \}$$

имеют вид

$$\begin{array}{ll} w & w^r \\ 011 & 110 \\ 0111 & 1110 \end{array}$$

Таким образом, использование степеней и операции обращения позволят задавать такие конфигурации цепочек, которые не являются регулярными.

5.2. ОПРЕДЕЛЕНИЕ МП-АВТОМАТА

МП-автомат задается четырьмя атрибутами (объектами):

- 1) алфавитом входных символов, обычно в него включают концевой маркер $\wedge$;
- 2) алфавитом магазинных символов, включающим маркер дна магазина $\boxminus$;
- 3) алфавитом состояний, включающим начальное состояние;
- 4) управляющим устройством, которое комбинации:
 - входного символа;
 - магазинного символа;
 - символа состояния

ставит в соответствие ВЫХОД и/или ПЕРЕХОД.

ВЫХОД означает завершение работы МП-автомата. Проводя аналогию, можно сказать, что ВЫХОД является аналогом функции выхода конечного распознавателя.

ПЕРЕХОД выполняет действие над магазином, состоянием и входом и является в некотором смысле аналогом функции переходов конечного распознавателя.

Обычно ПЕРЕХОД включает следующие действия-команды:

ВЫТОЛКНУТЬ	– верхний символ выталкивается из магазина;
ВТОЛКНУТЬ(A)	– в магазин вталкивается символ A ;
СОСТОЯНИЕ(S)	– следующим текущим состоянием становится S ;
СДВИГ	– осуществляется просмотр следующего символа во входные цепочки;
ДЕРЖАТЬ	– текущий символ входной цепочки остается прежним.

МП-автомат начинает работу с некоторого состояния магазина, называемого начальным. Обычно оно соответствует пустому содержимому магазину.

Кроме того, также должно быть определено начальное состояние управляющего устройства.

В МП-автомате запрещены операции, которые запрашивают входной символ после концевого маркера и выталкивают маркер дна магазина.

Обычно при задании МП-автомата используются сокращенные названия команд: ВТОЛК(A), ВЫТОЛК, СОСТ(S) ДЕРЖ, ОТВЕРГ, ДОПУСТ.

Если ВЫХОД МП-автомата имеет два состояния ДОПУСТИТЬ и ОТВЕРГНУТЬ, то такой МП-автомат называется МП-распознавателем.

Задается МП-автомат совокупностью решающих таблиц по числу состояний автомата. В решающей таблице каждому символу входной строки и символу вершины магазина ставится в соответствие значение ВЫХОДА и ПЕРЕХОДА. Совокупность таких решающих таблиц задает полное описание автомата. Решающую таблицу в некотором смысле можно рассматривать как аналог таблицы переходов/выходов конечного автомата.

В дальнейшем о задании МП-автомата в виде совокупности решающих таблиц будем говорить, как о стандартные представления МП-автомата.

Рассмотрим в качестве примера МП-автомат, выполняющий разбор скобок в арифметических выражениях. Данный МП-автомат задается следующим образом:

- 1) входное множество $\{ (,), ^ \}$;
- 2) множество магазинных символов $\{ A, \square \}$;
- 3) множество состояний $\{ S \}$, где S – начальное состояние;
- 4) переходы

Текущие значения входа, магазина и состояния	ПЕРЕХОД
$(, A, S$	ВТОЛК(A), СОСТОЯНИЕ(S), СДВИГ
$(, \square, S$	ВТОЛК(A), СОСТОЯНИЕ(S)
$), A, S$	ВЫТОЛК(A), СОСТОЯНИЕ(S), СДВИГ
$), \square, S$	ОТВЕРГНУТЬ
$^, A, S$	ОТВЕРГНУТЬ
$^, \square, S$	ДОПУСТИТЬ

- 5) начальное состояние магазина – магазин пуст ($\varnothing$);
- 6) МП-автомат имеет одно состояние S , оно же и начальное.

Данный МП-автомат задается следующей решающей таблицей:

S	Входной алфавит		
Алфавит магазина	(	)	$\wedge$
A	ВТОЛК(A) СДВИГ	ВЫТОЛК(A) СДВИГ	ОТВЕРГ
$\varnothing$	ВТОЛК(A) СДВИГ	ОТВЕРГ	ДОПУС

Теперь рассмотрим обработку цепочки $((()))$ таким МП-автоматом, которая пошагово представлена ниже:

№ п/п	Магазин	Состоя- ние	Остаток входной цепочки
1	$\varnothing$	$[S]$	$((()))^\wedge$
2	$\varnothing A$	$[S]$	$()())^\wedge$
3	$\varnothing AA$	$[S]$	$)())^\wedge$
4	$\varnothing A$	$[S]$	$())^\wedge$
5	$\varnothing AA$	$[S]$	$))^\wedge$
6	$\varnothing A$	$[S]$	$)^\wedge$
7	$\varnothing$	$[S]$	$^\wedge$
Допустить			

Рассмотрим другой пример МП-автомата с двумя состояниями, разбирающего цепочки множества

$$\{0^n 1^n \mid n > 0\}.$$

Определение МП-автомата имеет вид:

- 1) входное множество $\{0, 1, \wedge\}$;
- 2) множество магазинных символов $\{Z, \varnothing\}$;
- 3) множество состояний $\{S_1, \}$, где S_1 – начальное состояние;

4) переходы:

– при обработке входного символа 0 в магазин вталкивается символ Z , МП-автомат находится в состоянии S_1 – фаза 1;

– при обработке входного символа 1 из магазина выталкивается символ Z . Автомат переходит в состояние S_2 – фаза 2.

Поскольку определены две фазы работы МП-автомата, фаза 1 вталкивания символа Z в магазин, соответствующая состоянию $[S_1]$, и фаза 2 – выталкивания, соответствующая состоянию $[S_2]$, то для каждой из фаз сконструируем собственные решающие таблицы.

Фаза 1 $[S_1]$	Входной алфавит		
Алфавит магазина	0	1	$\wedge$
Z	СОСТОЯНИЕ (S_1) ВТОЛК(Z) СДВИГ	СОСТОЯНИЕ (S_2) ВТОЛК(Z)	ОТВЕРГ
$\varnothing$	СОСТОЯНИЕ (S_1) ВТОЛК(Z) СДВИГ	ОТВЕРГЪ	ОТВЕРГ

Фаза 2 $[S_2]$	Входной алфавит		
Алфавит магазина	0	1	$\wedge$
Z	ОТВЕРГ	СОСТОЯНИЕ (S_2) ВЫТОЛК(Z) СДВИГ	ОТВЕРГ
$\varnothing$	ОТВЕРГ	ОТВЕРГ	ДОПУСТИТЬ

Рассмотрим процесс разбора цепочки 000111:

№ п/п	Магазин	Состояние	Остаток входной цепочки
1	$\varnothing$	$[S_1]$	000111 $\wedge$
2	$\varnothing Z$	$[S_1]$	00111 $\wedge$

3	⊗ZZ	[S ₁]	0111 ^
4	⊗ZZZ	[S ₁]	111 ^
5	⊗ZZ	[S ₂]	11 ^
6	⊗Z	[S ₂]	1 ^
7	⊗	[S ₂]	^
Допустить			

Цепочка 000111 допускается МП-автоматом.
Рассмотрим теперь разбора цепочки 0001011:

№	Магазин	Состоя- ние	Остаток входной це- почки
1	⊗	[S ₁]	0001011 ^
2	⊗Z	[S ₁]	001011 ^
3	⊗ZZ	[S ₁]	01011 ^
4	⊗ZZZ	[S ₁]	1011 ^
5	⊗ZZ	[S ₂]	011 ^
Отвергнуть			

Цепочка 0001011 не допускается МП-автоматом.

5.3. РАСШИРЕНИЕ ОПЕРАЦИЙ НАД МАГАЗИНОМ

МП-автоматы, у которых операции над магазином выполняются только над одним верхним символом типа ВЫТОЛКНУТЬ и ВТОЛКНУТЬ, называются примитивными. Однако при реализации эти операции являются слишком мелкими, поэтому часто их укрупняют. Более того, анализ состояния магазина можно расширить на несколько символов. Разберем данную модификацию операций.

Введем расширенную операцию ЗАМЕНИТЬ над магазином. Эта операция заключается в выталкивании верхнего символа магазина и выполнении затем вталкивания нескольких символов. Операция ЗАМЕНИТЬ(ABC) осуществляет следующие действия:

- 1) выталкивает верхний символ магазина;
- 2) вталкивает в магазин последовательность символов ABC.

Проиллюстрируем операцию на примере разбора цепочек множества $\{0^n 1^n \mid n > 0\}$.

Определение МП-автомата:

- 1) входное множество $\{0, 1, \wedge\}$;
- 2) алфавит магазинных символов $\{X, Z, \square\}$;
- 3) число состояний одно, в решающей таблицы опускаем его;
- 4) решающая таблица:

Алфавит магазина	Входной алфавит		
	0	1	$\wedge$
X	ЗАМЕН (XZ) СДВИГ	ВЫТОЛК ДЕРЖАТЬ	ОТВЕРГ
Z	ОТВЕРГ	ВЫТОЛК СДВИГ	ОТВЕРГ
$\square$	ОТВЕРГ	ОТВЕРГ	ДОПУСТ

- 5) начальное состояние магазина X.

Покажем разбор цепочки 000111 МП-автоматом:

№ п/п	Состоя- ние ма- газина	Остаток входной цепочки	Действия
1	$\square X$	000111 $\wedge$	ЗАМЕНИТЬ (XZ), СДВИГ
2	$\square ZX$	00111 $\wedge$	ЗАМЕНИТЬ (XZ), СДВИГ
3	$\square ZZX$	0111 $\wedge$	ЗАМЕНИТЬ (XZ), СДВИГ
4	$\square ZZZX$	111 $\wedge$	ВЫТОЛКНУТЬ, ДЕРЖАТЬ
5	$\square ZZZ$	111 $\wedge$	ВЫТОЛКНУТЬ, СДВИГ
6	$\square ZZ$	11 $\wedge$	ВЫТОЛКНУТЬ, СДВИГ
7	$\square Z$	1 $\wedge$	ВЫТОЛКНУТЬ, СДВИГ
8	$\square$	$\wedge$	Допустить

При обработке нулей в вершине магазина всегда находится символ X . При смене входного символа с 0 на 1 происходит выталкивание символа X без смены текущего символа, а далее разбор осуществляется обычным образом.

5.4. ПЕРЕВОД ВХОДНЫХ ЦЕПОЧЕК

Дополним операции над состоянием, магазином и входом следующей операцией над выходом, по которой на выход МП-автомата выдается цепочка ABC. Команда имеет следующий формат:

ВЫДАТЬ (ABC).

Проиллюстрируем использование операции **ВЫДАТЬ** на примере.

Пусть на вход МП-автомата поступают цепочка из произвольно расположенных m нулей (0) и n единиц (1). Требуется преобразовать ее к новому виду $1^n 0^m$, выполнив перестановки символов.

Организуем работу МП-автомата следующим образом. Если во входной последовательности встречается единица, то она передается на выход. Все нули запоминаются в магазине. По прочтению концевого маркера входной строки нули выталкиваются на выход.

В МП-автомате используется одно состояние, поэтому он представлен следующей решающей таблицей

Алфавит магазина	Входной алфавит		
	0	1	$\wedge$
0	ВТОЛКНУТЬ(0) СДВИГ	ВЫДАТЬ(1) СДВИГ	ВЫТОЛКНУТЬ ВЫДАТЬ(0)
$\boxtimes$	ВТОЛКНУТЬ(0) СДВИГ	ВЫДАТЬ(1) СДВИГ	ЗАВЕРШИТЬ

Разбор входной цепочки 01011 показан в таблице:

№ п/п	Состояние магазина. Операция	Остаток входной цепочки	Выходная цепочка
1	□ ВТОЛК(0) СДВИГ	<u>0</u> 1011 [^]	
2	□0 ВЫДАТЬ(1) СДВИГ	<u>1</u> 011 [^]	1
3	□0 ВТОЛК(0) СДВИГ	<u>0</u> 11 [^]	1
4	□00 ВЫДАТЬ(1) СДВИГ	<u>1</u> 1 [^]	11
5	□00 ВЫДАТЬ(1) СДВИГ	<u>1</u> [^]	111
6	□00 ВЫДАТЬ(0) ВЫТОЛК	<u>^</u>	1110
7	□0 ВЫДАТЬ(0) ВЫТОЛК	<u>^</u>	11100
8	□ Допустить	<u>^</u>	

В МП-автоматах возможны заикливания. Заикливания могут быть выражены явно и тогда их достаточно просто выявить. Но заикливания могут быть также и неявными, скрытыми. Выявление неявных заикливаний требует уже некоторых дополнительных усилий. Поэтому всегда при конструировании МП-автомата следует иметь в виду данное обстоятельство и проводить тщательный его анализ на предмет выявления заикливаний и их устранение.

Пример явного заикливания МП-автомата приводится далее

Фрагмент решающей
таблицы

Алфавит мага- зина	Входной алфавит
	0
А	ВТОЛКНУТЬ(В) ДЕРЖАТЬ
В	ВЫТОЛКНУТЬ ДЕРЖАТЬ

Фрагмент разбора

№ п/п	Состоя- ние мага- зина	Остаток вход- ной цепочки
i	A	0 1
i+1	AB	0 1
i+2	A	0 1
j	AB	0 1
j+1	A	0 1

Из примера видно, что при обработке входного символа, если в вершине магазина МП-автомата находится символ А, то МП-автомат входит в закливание с удержанием входного символа 0.

6. ПРИМЕРЫ ОБЛАСТЕЙ ПРИМЕНЕНИЯ И РЕАЛИЗАЦИИ РАСПОЗНАВАТЕЛЕЙ

Задачу распознавания символьных последовательностей в общем виде можно сформулировать следующим образом. Пусть задан алфавит A и множество образованных из его символов всевозможных цепочек (слов) A^+ . Также пусть задано непустое множество слов L , такое, что $L \subset A^+$, и существует разбирающий язык L распознаватель. Разбор означает, что для некоторого слова $x \in A^+$ распознаватель определяет его принадлежность множеству L .

Такая задача зачастую называется задачей идентификации, а решающий ее распознаватель – идентифицирующим автоматом. Поэтому идентифицирующий автомат и распознаватель являются в контексте обработки символьных последовательностей понятиями-синонимами, хотя в других ситуациях это может быть и не так.

В настоящей главе будут разобраны типовые подходы к конструированию подобных идентифицирующих автоматов – распознавателей для различных задач.

6.1. МЕТОД РОЛЕЙ

Метод ролей проиллюстрируем на примере конструирования идентифицирующего автомата, определяющего правильность написания вещественных чисел без знака в языке программирования, например, в языке Бейсик.

Примеры правильного написания таких чисел выглядят следующим образом:

12E+08; .12E-10; 12E8; 12; .12; 12.5.

Как это видно, вещественные числа представляются несколькими способами записи (форматами). Первые три числа являются правильными записями в формате с плавающей точкой, а следующие три – в формате с фиксированной точкой.

Поскольку нас интересует правильность написания чисел безотносительно к их конкретным значениям, то повысим уровень абстракции входного алфавита распознавателя и произведем его

перекодировку. Поставим в соответствие десятичным цифрам алфавита символ ц (аббревиатура *цифры*); знакам $-$ и $+$ — *знак*; символы точки и порядка оставим без изменений и также введем их в новый алфавит. После этого входной алфавит распознавателя примет вид

$$\{\text{ц, знак, E, .}\}.$$

Осуществим переход от вербального описания автомата к его таблице переходов, для чего определим те роли, которые играют символы построенного алфавита в форматах записи. Такие перенумерованные роли перечислены далее:

- 1 — цифра, стоящая до десятичной точки;
- 2 — необязательная десятичная точка;
- 3 — цифра, стоящая после десятичной точки;
- 4 — символ E;
- 5 — знак порядка;
- 6 — цифра порядка;
- 7 — обязательно присутствующая десятичная точка, требующая десятичной цифры.

Свяжем роли букв входного алфавита с позициями форматов и продемонстрируем это на конкретных числах. В приводимых далее записях чисел под каждой цифрой проставлена ее роль в данной позиции записи. Верхний ряд чисел представлен в формате с фиксированной точкой, а нижний — с плавающей.

1 2 5	1 2 5 . 5	. 1 2 5	
1 1 1	1 1 1 2 3	7 1 1 1	
1 2 . 5 E + 5	1 2 5 E + 5	1 2 5 E 5	1 2 5 E 5
1 1 2 3 4 5 6	1 1 1 4 5 6	1 1 1 4 6	1 1 1 4 6

При определении ролей важно рассмотреть и охватить всевозможные форматы записи.

После выявления ролей нужно перейти к следующему этапу конструирования идентифицирующего автомата — построению таблицы переходов. Для этого поставим в соответствие каждой роли одно состояние автомата, введем начальное состояние S и состояние ошибки F (соответствует неправильной записи числа).

Построим пустую таблицу переходов вида:

Со- стояния	Входные символы				Y/N
	ц	Е	▪	Знак	
S					
1					
2					
3					
4					
5					
6					
7					
F					

Далее, начиная с начального состояния, определяем частичные функции переходов (заполняем клетки таблицы) для каждой строки и состоянию приписываем признак допускающего или недопускающего состояния (Y/N). При этом параллельно моделируем процесс просмотра входной цепочки по всем форматам и для всех входных букв.

Например, для начального состояния S фрагмент таблицы переходов будет выглядеть следующим образом:

Со- стояния	Входные символы				Y/N
	ц	Е	▪	Знак	Y\N
S	1	F	7	F	N
1					
2					
...					

7					
F					

Далее процесс вычисления частичных функций переходов последовательно продолжаем для состояний 2, 3 ... и т.д. до тех пор, пока не будут исчерпаны все состояния.

Теперь построенная таблица переходов автомата примет следующий вид:

Со- стояния	Входные символы				Y/N
	ц	Е	▪	Знак	Y\N
S	1	F	7	F	N
1	1	4	2	F	Y
2	3	4	F	F	Y
3	3	4	F	F	Y
4	6	F	F	5	N
5	6	F	F	F	N
6	6	F	F	F	Y
7	3	F	F	F	N
F	F	F	F	F	N

Далее (рис.41) приводится диаграмма состояний автомата, соответствующая построенной таблице переходов.

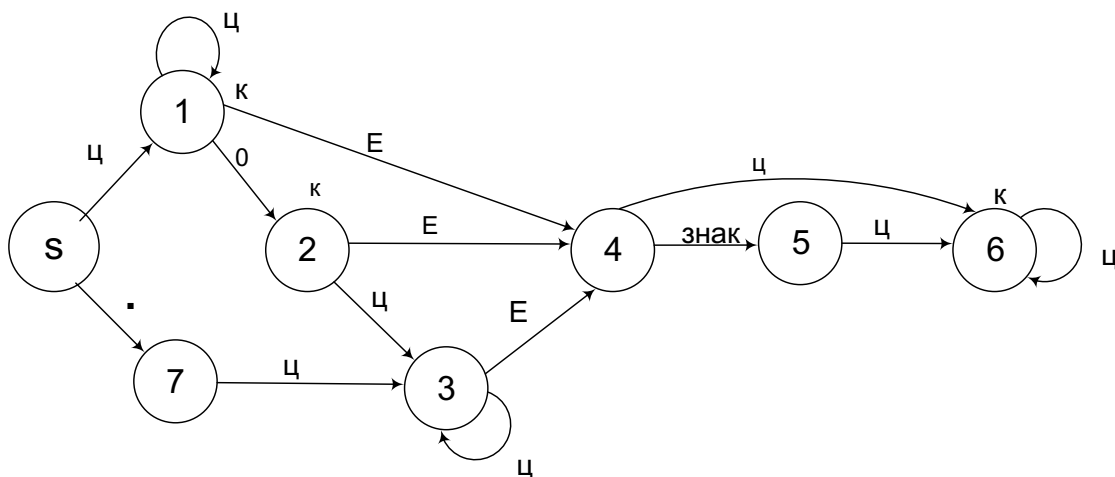


Рис.41. Диаграмма состояний идентифицирующего автомата для определения правильности написания вещественных чисел

Для упрощения в диаграмме состояний переходы в состояние ошибки F не показаны, а подразумеваются.

6.2. ПОСТРОЕНИЕ ИДЕНТИФИЦИРУЮЩЕГО АВТОМАТА МЕТОДОМ ПРЕФИКСОВ

Построение основано на введении для каждой подцепочки, которая может выступать префиксом в другой подцепочке, отдельного состояния. Проиллюстрируем метод примером.

Пусть задан входной алфавит {Б, Ы, Т, Л, А, К, О, С, У, $\wedge$ }, где $\wedge$ – маркер конца цепочки, и распознаваемое множество {БЫТ, БЫК, БЫЛ, БАК, БАКУ, КОСА, ОСА}. Построение таблицы переходов автомата начинаем с введения в него первого начального состояния (первая строка), оно соответствует случаю, когда на вход автомата подана пустая цепочка λ . Далее впишем в соответствующие клетки первой строки таблицы состояния Б, К, О, поскольку их имена являются начальными символами префиксов слов из распознаваемого множества, получим следующую таблицу переходов:

Состояния	Входные символы									
	Б	Ы	Т	Л	А	К	О	С	У	$\wedge$
λ	Б				К	О				

Для состояний Б, К и О заведем новые строки в таблице переходов и также заполним их новыми состояниями, для которых состояния Б, К и О являются префиксами:

Состояния	Входные символы									
	Б	Ы	Т	Л	А	К	О	С	У	^
λ	Б				К	О				
Б		БЫ			БА					
К							КО			
О								ОС		

Процесс продолжим до тех пор, пока вновь образуемые состояния не будут представлять слова распознаваемого множества. Итоговая таблица переходов для примера выглядит так:

Состояния	Входные символы									
	Б	Ы	Т	Л	А	К	О	С	У	^
λ	Б				К	О				
Б		БЫ			БА					
К							КО			
О								ОС		
БЫ			БЫТ	БЫЛ		БЫК				
БА						БАК				
КО								КОС		
ОС		ОСЫ								
БЫТ										«БЫТ»
БЫЛ										«БЫЛ»
БЫК										«БЫК»
БАК									БАКУ	«БАК»
КОС					КОСА					
ОСЫ										«ОСЫ»
БАКУ										«БАКУ»
КОСА										«КОСА»

Последний столбец таблицы переходов соответствует конечному маркеру и показывает, каким словам распознаваемого множества соответствует данное состояние. Здесь распознанные слова заключены в кавычки, а пустые элементы таблицы указывают на состояние ошибки (не распознано). Отметим, что представление идентифицирующего автомата подобной таблицей ввиду ее разреженности требует достаточно больших затрат памяти. Особенно это свойство проявляется, когда распознаваемое множество содержит большое число длинных слов.

Другой способ задания функции переходов – ее представление в виде связанного списка. Реквизиты (структура) элемента такого списка имеют следующий вид:

Текущее состояние	Входные буквы	Указатели новых состояний
-------------------	---------------	---------------------------

Построение списка переходов выполним на примере рассмотренного распознаваемого множества. Так, для первого состояния элемент списка будет иметь следующий вид:

Текущее состояние	Входные буквы	Указатели новых состояний
1	Б	2
	К	3
	О	4
	ОШИБКА	

Здесь первый реквизит списка представляет текущее состояние, второй – возможные входные буквы, третий – новые состояния. ОШИБКА – это все остальные входные буквы, приводящие в состояние ошибки. Далее для состояний 2, 3, 4 строим аналогичные элементы и продолжаем процесс до завершения, после чего окончательный список переходов принимает следующий вид:

Текущее состояние	Входные буквы	Указатели новых состояний	
1	Б	2	Б...
	К	3	К...
	О	4	О...
	ОШИБКА		
2	Ы	5	БЫ...
	А	6	БА...
	ОШИБКА		
3	О	7	КО...
	ОШИБКА		
4	С	8	ОС...
	ОШИБКА		
5	Т	9	БЫТ...
	Л	10	БЫЛ...
	К	11	БЫК...
	ОШИБКА		
6	К	12	БАК...
	ОШИБКА		
7	С	13	КОС...
	ОШИБКА		
8	Ы	14	ОСЫ...
	ОШИБКА		
9	^		«БЫТ»
	ОШИБКА		
10	^		«БЫЛ»
	ОШИБКА		
11	^		«БЫК»
	ОШИБКА		
12	У	15	БАКУ...
	^		«БАК»
	ОШИБКА		
13	А	16	КОСА...
	ОШИБКА		
14	^		«ОСЫ»

	ОШИБКА		
15	^		«БАКУ»
	ОШИБКА		
16	^		«КОСА»
	ОШИБКА		

В качестве комментария рядом с новым состоянием написан разбираемый префикс. Многоточие указывает на продолжение распознавания. Слово, заключенное в кавычки, означает, что слово идентифицировано.

Итак, представление автомата в виде списка переходов является компактным в сравнении с табличным, однако если возникнет потребность в расширении распознаваемого множества, то в данной нотации построение списка переходов нужно выполнить заново, что представляет существенный недостаток.

В реальной практике проектирования распознавателей случаи расширения распознаваемого множества не так уж редки. Более того, расширение часто приходится выполнять в процессе распознавания. В этом случае необходимо иметь возможность расширять список переходов, для чего потребуются иная нотация функции переходов. Рассмотрим и поясним ее далее на примере рассмотренного распознаваемого множества.

1	Б	2
	Ы	4
	Т	5
	^	
	«БЫТ»	
2	К	3
	О	
	С	
	А	
	^	
	«КОСА»	
3	О	
	С	

	А	
	^	
	«ОСА»	
4	А	
	К	
	^	7
	«БАК»	
5	К	6
	^	
	«БЫК»	
6	Л	
	^	
	«БЫЛ»	
7	У	
	^	
	«БАКУ»	

В такой нотации список задается явно в виде последовательности состояний (1, 2, ..., 7), при этом каждое состояние сопоставляется одному слову распознаваемого множества (записывается в кавычках внизу второго реквизита состояния), а к каждой букве приписывается альтернатива, которая определяет переход к следующему состоянию, если сканируемая текущая буква не совпадает с данной.

Теперь необходимо расширить распознаваемое множество словом «КОЛБАСА». Создаем новое состояние 8, дописываем его в конец списка и вносим изменения во второе состояние. После этого расширенная таблица переходов примет вид:

1	Б	2
	Ы	4
	Т	5
	^	
	«БЫТ»	
2	К	3
	О	

	С	8
	А	
	^	
	«КОСА»	
3	О	
	С	
	А	
	^	
4	А	
	К	
	^	7
	«БАК»	
5	К	6
	^	
	«БЫК»	
6	Л	
	^	
	«БЫЛ»	
7	У	
	^	
	«БАКУ»	
8	Л	
	Б	
	А	
	С	
	А	
	^	
	«КОЛБАСА»	

Хотя данный способ расширения распознаваемого множества является простым, но при многократных расширениях список будет сильно разрастаться.

6.3. НЕАВТОМАТНОЕ РЕШЕНИЕ ЗАДАЧИ ИДЕНТИФИКАЦИИ

Существуют методы идентификации, отличные от автоматных. Идея заключается в том, чтобы распознаваемое множество хранилось в виде таблицы или связанного списка, а входное слово сравнивалось с элементами такой структуры хранения по типу справочника. Расширение распознаваемого множества в этом случае никаких затруднений не вызывает.

6.3.1. Метод линейного поиска

Этот метод часто называют линейным поиском, и его суть заключается в том, что входное слово последовательно сравнивается с элементами распознаваемого множества. Хранение множества допускается либо в виде последовательного массива либо в виде связанного списка.

6.3.2. Метод индексов

Метод индексов иначе называется прямым доступом. Пусть задано распознаваемое множество, состоящее из N элементов, и какая-либо структура хранения (таблица, связанный список). Нумерация последовательных элементов этой структуры образует ее адресное пространство. Если вложить элементы распознаваемого множества в структуру хранения, то каждый из них получит свой адрес или, иначе, индекс.

В рассматриваемом методе между элементами множества и их индексами устанавливается некоторая функциональная связь вида

$$\text{индекс} = F(\text{распознаваемый элемент}).$$

Ее часто называют формулой рандомизации. Тогда доступ к соответствующему адресу может быть осуществлен путем ее вычисления. Например, пусть распознаваемое множество представляет картотеку сотрудников со следующими реквизитами отдельной карточки:

Табельный номер	ФИО	Прочие данные
--------------------	-----	------------------

Тогда формулу рандомизации можно реализовать простейшим образом:

$$\text{индекс} = \text{табельный номер}.$$

Однако на практике такие простейшие формулы применяются редко ввиду неэкономного расхода памяти.

В программировании при обработке текстовой информации ключевые поля явно могут не задаваться, тогда приходится устанавливать косвенные индексные зависимости. Например, в первых версиях языка программирования типа Бейсик идентификаторы задавались в виде <буква>|<буква><цифра>, где <буква> представляет все буквы латинского алфавита, а <цифра> – цифры от 0 до 9. Число букв в алфавите 26, цифр 9, следовательно, всего может быть не более 286 идентификаторов.

Рассмотрим задачу идентификации, когда в качестве распознаваемого множества выступает множество всех идентификаторов языка Бейсик. В качестве структуры хранения выберем таблицу и ее заполнение выполним из следующих соображений. Перенумеруем буквы алфавита (А – 1, В – 2 , ... , Z – 26), свяжем с каждым идентификатором индекс следующим образом:

индекс = *i* для односимвольных идентификаторов;

индекс = *i* + 26 * (*цифра* + 1) для двухсимвольных.

Здесь *i* – номер буквы алфавита. Тогда индекс для идентификатора А равен 1, для Z9 – 286.

Теперь идентификация осуществляется так: входное слово перекодируется в индекс и далее осуществляется сравнение входного слова с элементом таблицы, расположенным по полученному адресу.

Метод индексов требует наличия двух условий:

- фиксированного распознаваемого множества и однозначного его отображения на структуру хранения;
- простоты вычисления формулы рандомизации.

Кроме того, расширение множества является проблематичным.

6.4. ИДЕНТИФИКАЦИЯ ПО УПОРЯДОЧЕННОМУ СПИСКУ

Идентификация существенно сокращается, если распознаваемое множество упорядочено (отсортировано) по какому-либо признаку. В этом случае возможно применение различных видов поиска, среди которых одним из самых эффективных и простых является двоичный (бинарный, логарифмический) поиск.

6.4.1. Бинарный поиск

Рассмотрим бинарный поиск на следующем примере. Пусть распознаваемое множество представляет записи с ключами и упорядочено в порядке их возрастания. Тогда двоичный поиск заключается в последовательном делении отрезка чисел пополам и сужении интервала неопределенности.

При мощности множества в N элементов число шагов на поиск составляет величину порядка $\log N$.

Пусть распознаваемое множество – это ключевые слова языка программирования. Упорядочим их в лексикографическом порядке и разместим в таблице. Здесь бинарный поиск будет проводиться следующим образом:

Входное слово	Шаг	Ключевые слова
		1. ARRAY
		2. BEGIN
	2 —————>	3. DO
		4. ELSE
	3 —————>	5. END
	4 —————>	6. FOR
FOR	1 —————>	7. GOTO
		8. INTEGER
		9. PROCEDURE
		10. REAL
		11. THEN
		12. UNTIL
		13. WHILE

Если число элементов множества не кратно 2^n , то возникает вопрос, что считать серединой отрезка. Кроме того, расширение множества требует реорганизации таблицы.

Для поиска можно использовать связанный список в виде двоичного дерева. Его корень соответствует середине отрезка, ссылки указывают на середины половин этого отрезка и т.д.

Дерево можно расширять, добавляя новые элементы со стороны концевых узлов. Но в этом случае быстродействие поиска может существенно ухудшиться. Поэтому любое существенное расширение поискового пространства требует реорганизации дерева.

6.4.2. Метод расстановки

Этот способ является в некотором смысле комбинацией метода индексов и методов поиска по упорядоченным множествам и заключается в следующем. Все распознаваемое множество разбивается на отдельные подмножества, каждому из которых приписывается свой индекс. Подмножества могут размещаться в списках. Для всех списков составляется таблица указателей, которая индексируется. Идентификация осуществляется в два приема:

- 1) вычисляется по входному слову индекс и отыскивается соответствующий список;
- 2) выполняется поиск по списку и, если слово отсутствует, оно добавляется к списку.

0	→	FOR	→	BEGIN	→	THEN
1	→	GOTO				
2	→	INTEGER				
3	→	WHILE	→	REAL		
4						
5	→	DO	→	ARRAY	→	END
6	→	PROCEDURE				

В методе расстановки всегда приходится находить компромисс между длиной списков и размером индексируемой таблицы. Обычно стремятся иметь списки одинаковой длины.

6.5. ОБНАРУЖЕНИЕ СЛОВ С НЕЧЕТКИМИ ГРАНИЦАМИ

Границы слов, подлежащих идентификации, могут быть как четкими (например, в естественных языках слова в предложении разделяются пробелами), так и нечеткими. Например, следующую запись можно воспринимать как оператор присваивания в языке программирования Фортран и как оператор цикла в языке программирования Бейсик:

10 FORK = STOP

В случае Бейсика запись при вставке пробелов имеет вид:

10 FOR K = S TO P

Задачу идентификации в случае нечетких границ слов часто называют задачей обнаружения префиксов. Она достаточно просто решается при наложении следующего ограничения – в распознаваемом множестве ни один элемент не должен являться префиксом для другого элемента. В этом случае можно построить идентифицирующий автомат и в нем предусмотреть выход не по конечному маркеру, а по обнаружению префикса.

В случае не автоматного решения задачи следует воспользоваться следующим алгоритмом:

- 1) завести некоторую текстовую переменную CHAR и присвоить ей значение – пустая символьная строка;
- 2) из входной строки считать текущий символ и присоединить его к CHAR;
- 3) провести идентификацию CHAR с распознаваемым множеством. При неудаче повторить алгоритм с п.2.

Разобранные подходы не исчерпывают всего многообразия подходов к обнаружению и идентификацию слов с нечеткими границами, но как бы обозначают проблему в целом.

ЗАКЛЮЧЕНИЕ

Безусловно в учебном пособии изложены не все аспекты и подходы к символьной обработке информации, по данной тематике существует много разнообразных литературных источников, Их число постоянно увеличивается в связи с развитием прикладных приложений текстовой обработки. Но, все они в той или иной мере восходят своими корнями к ранним фундаментальным работам, которые заложили теоретический фундамент методов проектирования компиляторов. Как и все фундаментальные труды они до сих пор не только не потеряли своей актуальности и ценности, а даже перешли в разряд классических работ. Отдавая им должное, некоторые из них приведены в списке рекомендованной литературы к настоящему учебному пособию.

Само учебное пособие нужно рассматривать не как справочное руководство, а только как введение в предметную область распознавания символьных последовательностей. По части использования обратной польской записи для построения системы переходов авторы не нашли в доступном им пространстве литературные источники и поэтому привели на этот счет свои соображения.

РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА

1. Альфред, В. Компиляторы: принципы, технологии и инструментарий: пер. с англ. / Ахо А. [и др.]. – 2-е изд. М.: ООО «И.Д. Вильямс», 2008. – 1184 с.: ил.
2. Ахо, А. Теория синтаксического анализа, перевода и компиляции / А. Ахо, Дж. Ульман. – М.: Мир, 1978. – Т.1. – 612 с.
3. Ахо, А. Теория синтаксического анализа, перевода и компиляции / А. Ахо, Дж. Ульман. – М.: Мир, 1978. – Т.2. – 490 с.
4. Вирт, Н. Построение компиляторов: пер. с англ. / Н. Вирт. – М.: ДМК Пресс, 2010, –192 с.
5. Вишняков, Ю.М. Системное программирование. Конечные распознаватели: учеб пособие / Ю.М. Вишняков. – Таганрог: ТРТУ, 1991. – 74 с.
6. Грис, Д. Конструирование компиляторов для цифровых вычислительных машин / Д. Грис. – М.: Мир, 1975. – 545 с.
7. Донован, Дж. Системное программирование / Дж. Донован. – М.: Мир, 1975. – 540 с.
8. Кнут, Д. Искусство программирования – Т.1: Основные алгоритмы / Д. Кнут. – М.: Вильямс, 2002. – 720 с.
9. Кнут, Д. Семантика контекстно-свободных языков. / Д. Кнут // Семантика языков программирования. – М.: Мир, 1980. – 395 с.
10. Льюис, Ф. Теоретические основы проектирования компиляторов / Ф. Льюис, А. Розенкранц, Р. Стирнз. – М.: Мир, 1979. – 645 с.
11. Хантер, Р. Проектирование и конструирование компиляторов: пер. с англ. / Р. Хантер. – М.: Финансы и статистика, 1984. – 232 с.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. ОСНОВНЫЕ ПОЛОЖЕНИЯ ТЕОРИИ ФОРМАЛЬНЫХ ГРАММАТИК	5
1.1. Алфавит, цепочка, катенация	5
1.2. Грамматика, сентенциальная форма, язык.....	6
1.3. Понятие разбора, синтаксические деревья	10
1.4. Классификация языков	13
2. КОНЕЧНЫЕ РАСПОЗНАВАТЕЛИ.....	14
2.1. Детерминированный конечный распознаватель	14
2.2. Правосторонние автоматные грамматики	19
2.3. Переход от конечного распознавателя к правосторонним грамматикам	20
2.4. Квазиавтоматные грамматики и их приведение к правосторонним автоматным грамматикам	22
2.5. Минимизация детерминированных конечных распознавателей	25
2.6. Недетерминированный конечный распознаватель ...	35
2.7. Преобразование недетерминированного конечного распознавателя в детерминированный	37
3. РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ И КОНЕЧНЫЕ РАСПОЗНАВАТЕЛИ	43
3.1. Модифицированная форма бэкуса – наура и регулярные выражения	43
3.2. Система переходов	46
3.3. Преобразование системы переходов в недетерминированный конечный распознаватель	48
4. ПРЕОБРАЗОВАНИЕ РЕГУЛЯРНЫХ ВЫРАЖЕНИЙ НА ОСНОВЕ ОБРАТНОЙ ПОЛЬСКОЙ ЗАПИСИ.....	58
4.1. Основные понятия	58
4.2. Обратная польская запись регулярного выражения ..	61
4.3. Интерпретация вычисления опз регулярного выражения	64
5. АВТОМАТЫ С МАГАЗИННОЙ ПАМЯТЬЮ.....	77
5.1. Нерегулярные множества цепочек	77
5.2. Определение мп-автомата	78

5.3. Расширение операций над магазином	82
5.4. Перевод входных цепочек	84
6. ПРИМЕРЫ ОБЛАСТЕЙ ПРИМЕНЕНИЯ И РЕАЛИЗАЦИИ РАСПОЗНАВАТЕЛЕЙ	87
6.1. Метод ролей	87
6.2. Построение идентифицирующего автомата методом префиксов	91
6.3. Неавтоматное решение задачи идентификации	98
6.3.1. Метод линейного поиска	98
6.3.2. Метод индексов	98
6.4. Идентификация по упорядоченному списку	100
6.4.1. Бинарный поиск	100
6.4.2. Метод расстановки	101
6.5. Обнаружение слов с нечеткими границами	102
ЗАКЛЮЧЕНИЕ	103
РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА	104

Учебное издание

В и ш н я к о в Юрий Муссович

В и ш н я к о в Ренат Юрьевич

МЕТОДЫ РАСПОЗНАВАНИЯ СИМВОЛЬНЫХ
ПОСЛЕДОВАТЕЛЬНОСТЕЙ

Учебное пособие

Подписано в печать 27.12.2019.

Выход в свет 28.02.2020. Формат

Бумаги 60x84 1/16. Уч-изд. л. 5,3. Печать

Цифровая. Тираж 500 экз. Заказ №

Кубанский государственный университет
350020, г. Краснодар, ул. Ставропольская, 149

Издательско-полиграфический центр
Кубанского государственного университета
350020, г. Краснодар, ул. Ставропольская 149